



Introduzione alla code migration

Caso: Hadoop un middleware per applicazioni data-intensive

Prof. Michele Ciavotta

michele.ciavotta@unimib.it

Oltre il Data Passing: Introduzione alla Code Migration

Spesso si immagina la comunicazione nei sistemi distribuiti solo in termini di **scambio di dati**.

Questo modello presuppone che:

- il codice rimanga fisso su ciascun nodo;
 - solo input e output vengano trasmessi sulla rete.
-
- Esistono però scenari in cui, anziché spostare i dati, conviene **trasferire il codice**.

La **code migration** consiste nel trasferimento di un programma (o di una sua parte) da una macchina a un'altra.

Il codice può essere trasferito:

- prima dell'esecuzione;
- durante l'esecuzione, con migrazione dello stato.

Il codice diventa così un'entità mobile all'interno del sistema distribuito.

Oltre il Data Passing: Introduzione alla Code Migration

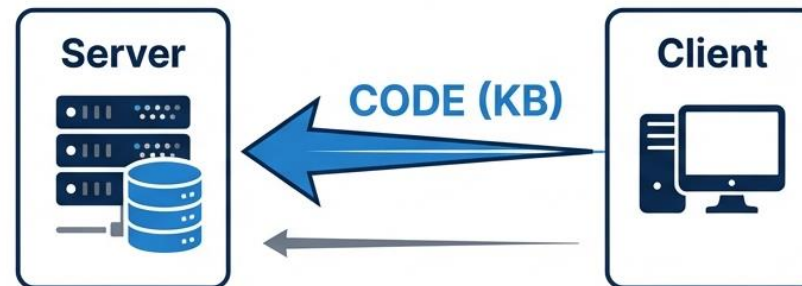
Ottimizzare le **prestazioni**:

- spostare i processi da macchine sovraccariche a macchine scariche (load distribution);
- minimizzare la comunicazione spostando il calcolo dove risiedono i dati;
- evitare il trasferimento di grandi moli di dati grezzi attraverso la rete.

Scenario A: Trasferimento Dati



Scenario B: Code Migration



Flessibilità

Nello sviluppo classico di applicazioni distribuite si decide a priori dove ciascun componente deve essere eseguito.

Se invece il codice può spostarsi tra le macchine:

- il sistema distribuito può essere **configurato dinamicamente**;
- le funzionalità possono essere installate solo quando servono;
- il binding client-server diventa più flessibile.

Scenario: Un server espone un file system.
Il protocollo è proprietario.

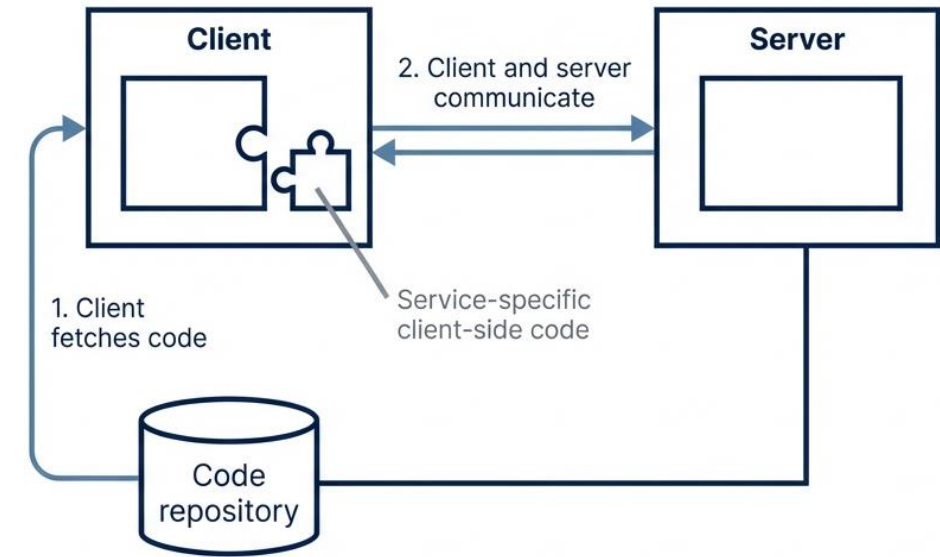
Approccio tradizionale: L'implementazione client del protocollo deve essere:

- Disponibile al momento dello sviluppo
- Linkata staticamente all'applicazione

Approccio dinamico: il client

- Scarica dinamicamente l'implementazione
- Inizializza il codice
- Invoca il server

Il repository del codice è tipicamente parte del server.



Il problema principale è la **sicurezza**:

- Il codice scaricato potrebbe essere malevolo
- Potrebbe accedere a risorse locali
- Potrebbe esfiltrare dati

Tipi di Mobilità

Weak Mobility (Mobilità Debole)



Weak mobility

Si trasferisce solo il **codice** (ed eventuali dati di inizializzazione); l'esecuzione riparte da zero sulla macchina di destinazione.

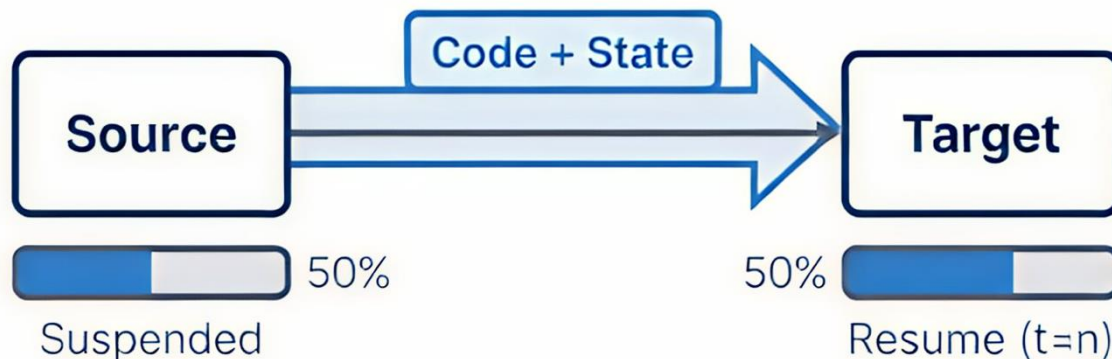
Esempio: applet Java, script scaricati dal browser.

Strong mobility

Si trasferiscono codice **e segmento di esecuzione**: il processo riprende sulla destinazione esattamente nello stato in cui era stato sospeso.

Esempio: process migration, live migration di VM.

Strong Mobility (Mobilità Forte)



Modelli di code migration – Mobilità debole

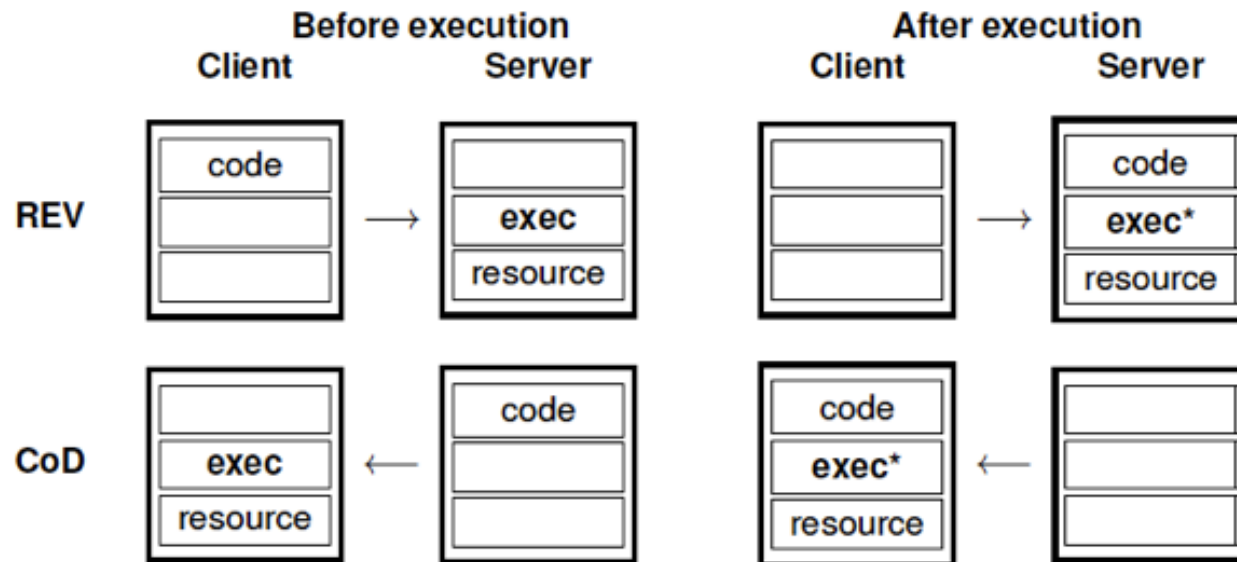
Weak Mobility (Mobilità Debole)



Nella **mobilità debole** si trasferisce solo il **code segment**: il programma riparte sempre dall'inizio sulla destinazione (es. Java applet). Semplice e portabile.

Esistono due varianti, che differiscono per **chi prende l'iniziativa**:

- **REV (Remote Evaluation)**: il client **invia** il proprio codice al server perché lo esegua usando le risorse del server.
- **CoD (Code on Demand)**: il client **scarica** codice dal server per eseguirlo localmente (es. JavaScript nel browser).



Nel diagramma, **exec*** indica il processo di esecuzione appena attivato dal codice ricevuto.

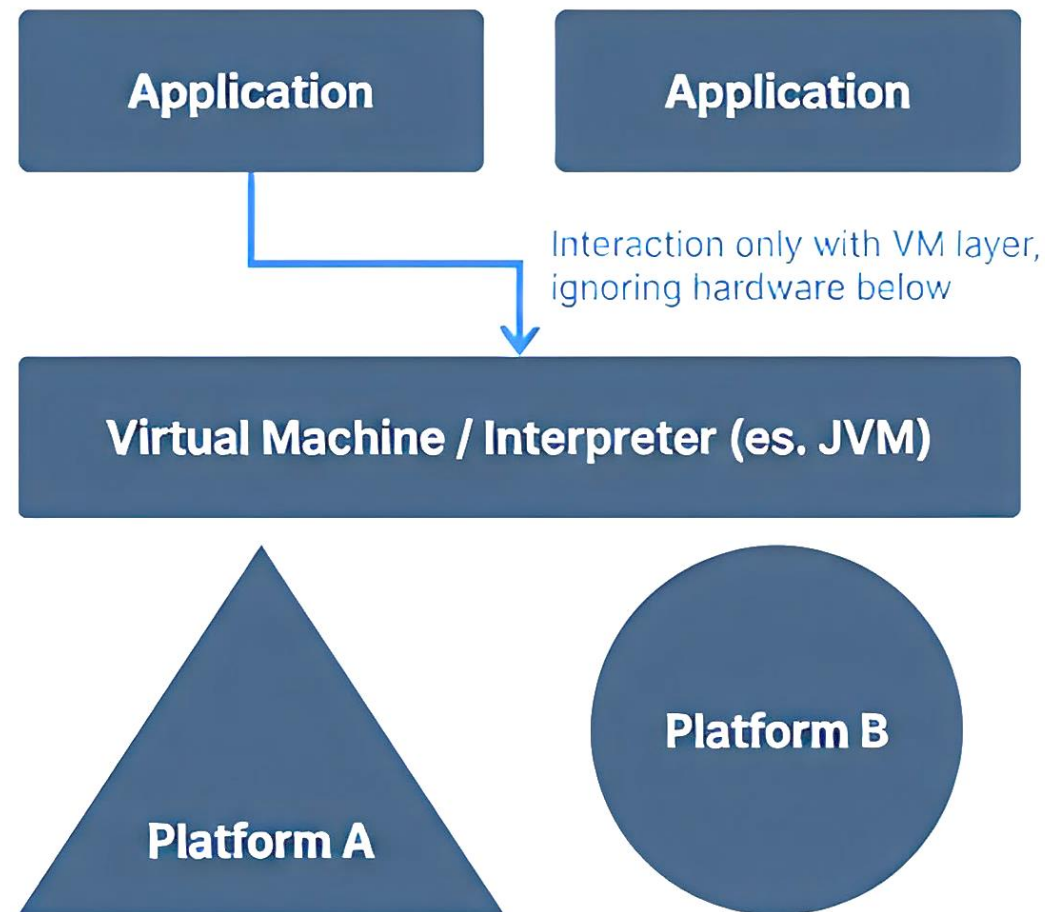
Problema dell'eterogeneità

Remote Evaluation (REV)

- Il client invia il codice al server per l'esecuzione.
- Sfrutta le risorse del server.
- Esempio: invio di uno script di ricerca a un database remoto.
- Modifica lo stato di esecuzione sul server.

Code on Demand (CoD)

- Il client richiede e scarica il codice dal server.
- Sfrutta le risorse locali del client.
- Esempio: JavaScript.
- Vantaggio: maggiore flessibilità e prestazioni, senza installazione permanente.

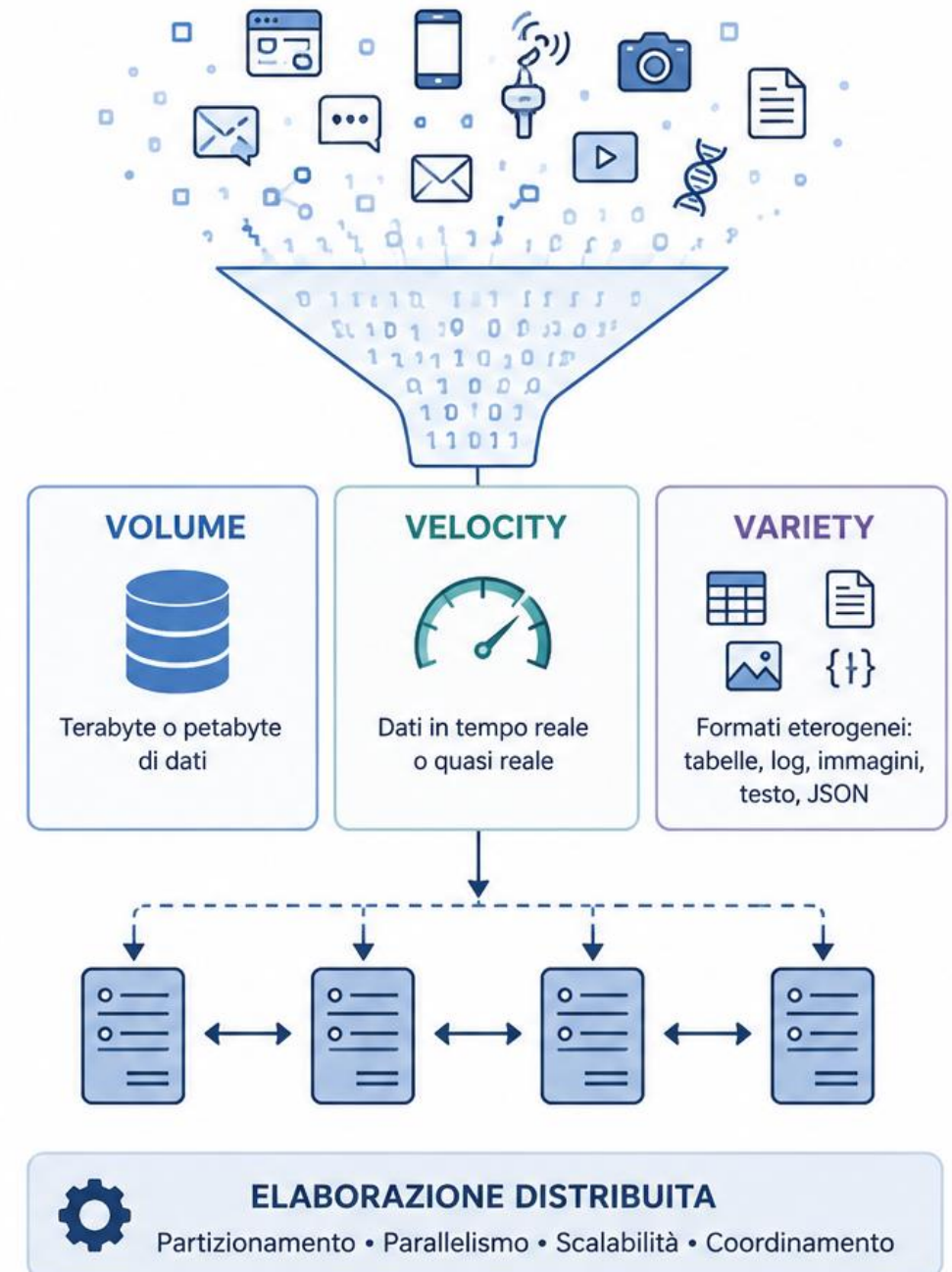


Un middleware per applicazioni data-intensive

APACHE HADOOP

Applicazioni Data-Intensive

- Sono applicazioni il cui collo di bottiglia non è la potenza di calcolo, ma il **volume, la velocità e la varietà dei dati** da elaborare
- Caratterizzate dalle **3 V**:
 - **Volume**: terabyte o petabyte di dati che non stanno su una singola macchina
 - **Velocity**: dati prodotti e da analizzare in tempo reale o quasi reale
 - **Variety**: formati eterogenei, da tabelle relazionali a log, immagini, testo, JSON
- Esempi tipici: indicizzazione web, analisi dei log, sistemi di raccomandazione, telemetria IoT, genomica, analisi di social network
- Conseguenza diretta: i dati vanno **partizionati ed elaborati in parallelo** su molte macchine — da qui la necessità di middleware, scalabilità e coordinamento distribuito

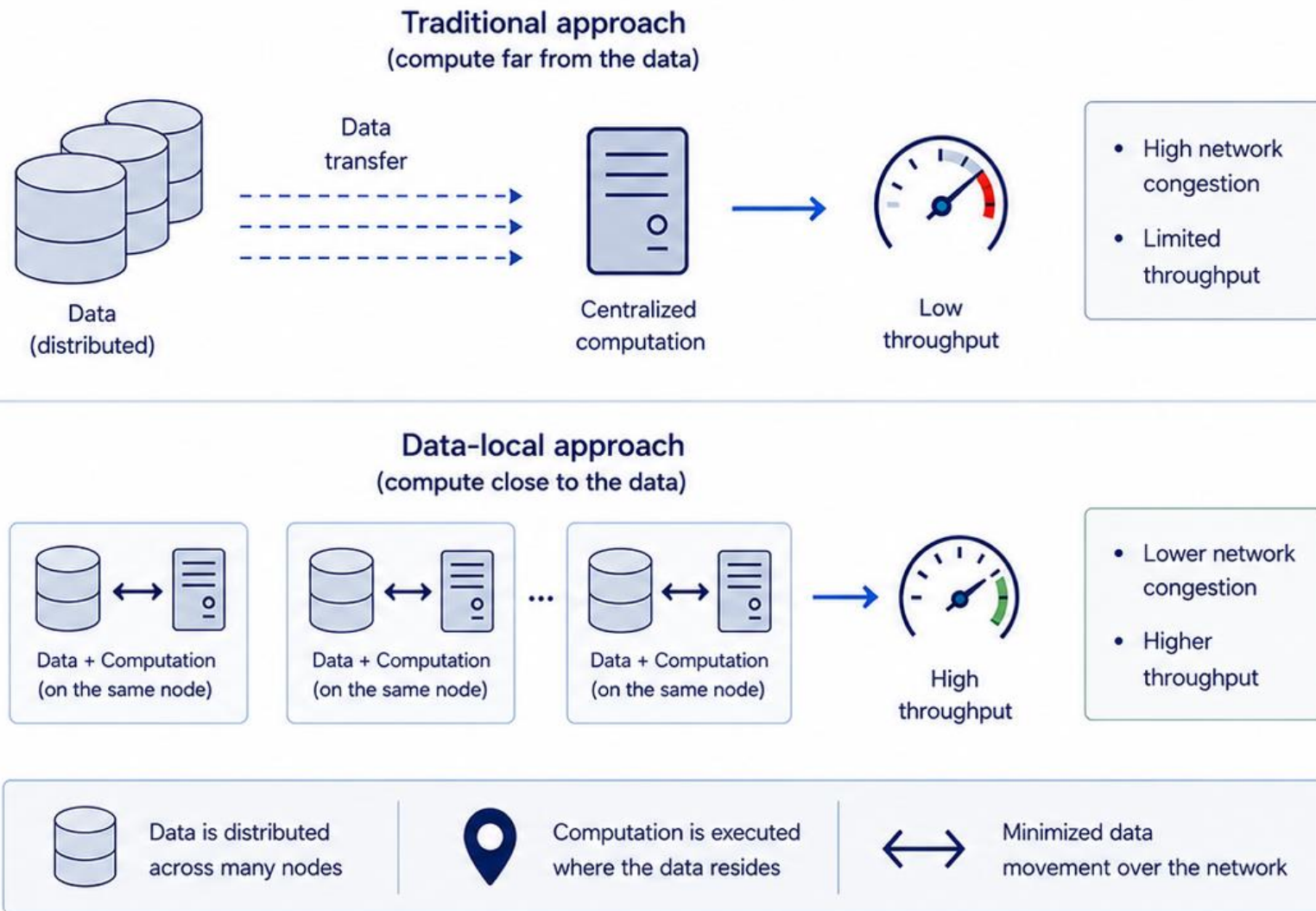


L'idea: portare la computazione vicino ai dati

Una computazione richiesta da un'applicazione è molto più **efficiente se viene eseguita vicino ai dati**

Questo è particolarmente vero quando la dimensione del dataset è enorme

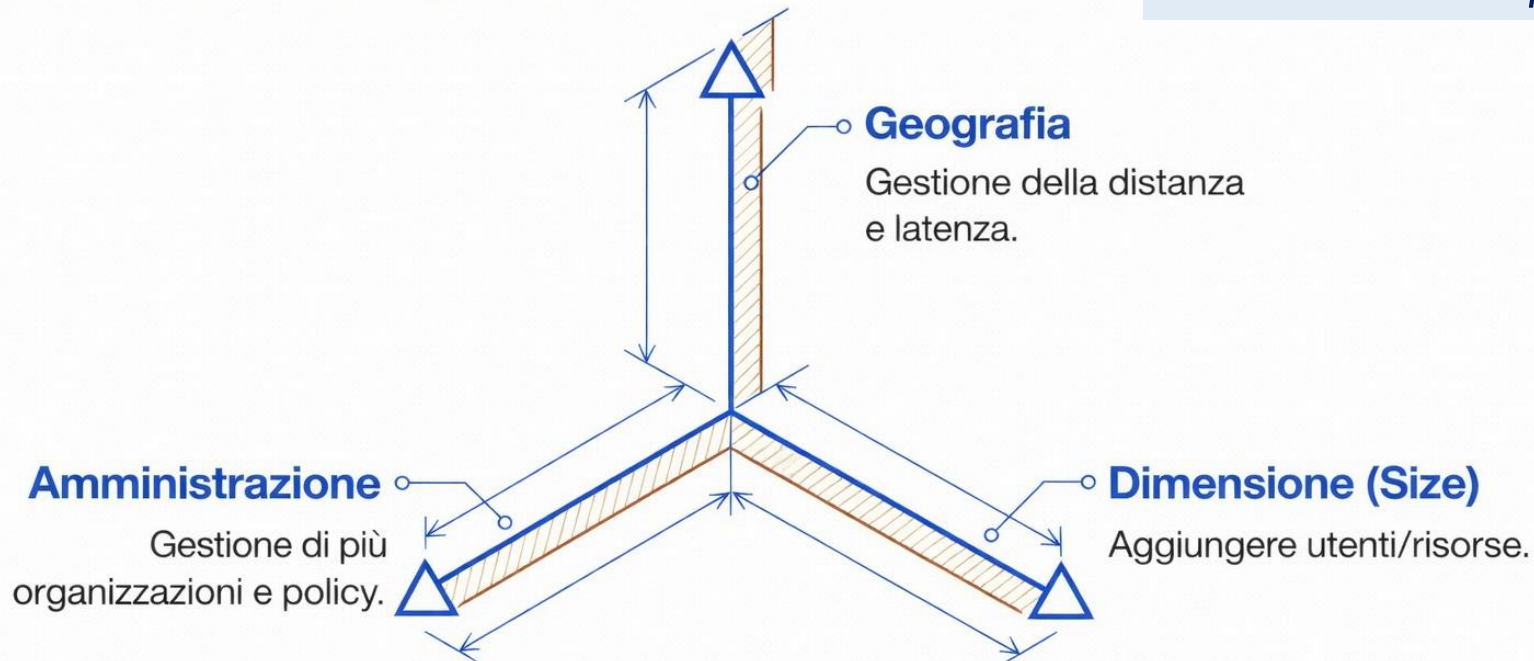
Questo **minimizza la congestione di rete** **aumenta il** complessivo **throughput** del sistema



Le Tre Dimensioni della Scalabilità

Classificazione di Neuman

Risponde alla domanda *"lungo quali assi un sistema può crescere?"*:



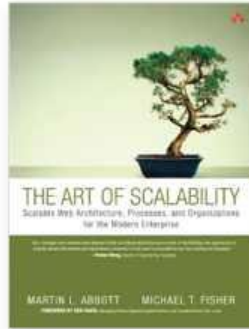
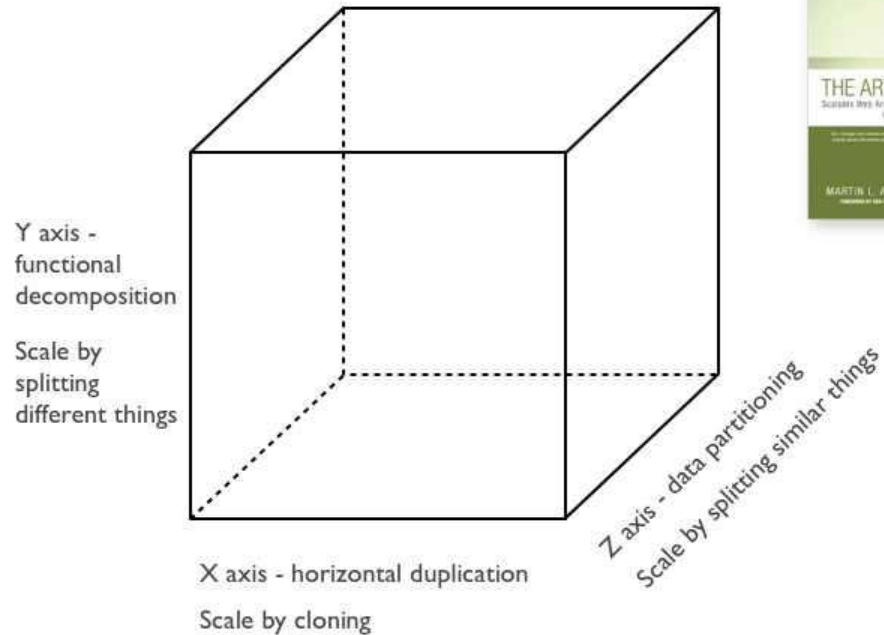
Scalabilità Dimensionale (Size scalability): Capacità di aggiungere facilmente più utenti e risorse senza degradare le prestazioni.

Scalabilità Geografica: Capacità del sistema di gestire utenti e risorse situati a grandi distanze, rendendo i ritardi di comunicazione (latenza) quasi impercettibili.

Scalabilità Amministrativa: Capacità di gestire il sistema anche quando si estende su diverse organizzazioni indipendenti con politiche differenti.

Scale Cube (Abbott & Fisher, The Art of Scalability)

3 dimensions to scaling



Risponde alla domanda "come replico/divido l'applicazione per scalare la dimensione di carico?"

X-axis: replicazione - eseguire più istanze di un'applicazione dietro un load balancer.

Y-axis: decomposizione funzionale - suddivisione dell'applicazione in più servizi distinti

Z-axis: partizionamento dei dati - ogni server è responsabile solo di un sottoinsieme dei dati

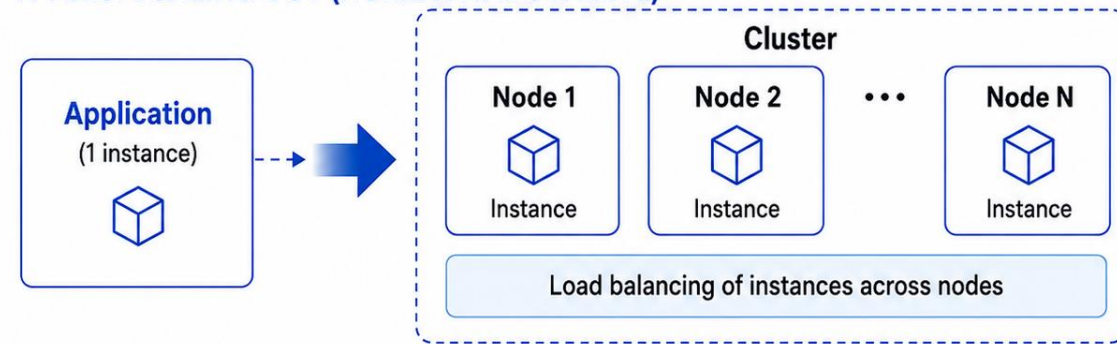
Hadoop abilita la scalabilità dell'applicazione

X-axis — Hadoop esegue più container che sono copie della stessa unità applicativa. Istanza e bilancia queste copie sui nodi del cluster.

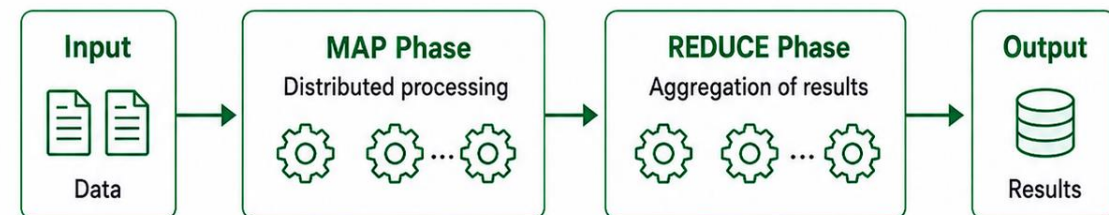
Y-axis — il modello di hadoop decompone l'applicazione in moduli funzionalmente diversi (la fase map e la fase reduce sono responsabilità distinte, con interfacce diverse).

Z-axis — lo split dei file in blocchi fa sì che ogni componente lavori solo su un sottoinsieme dei dati

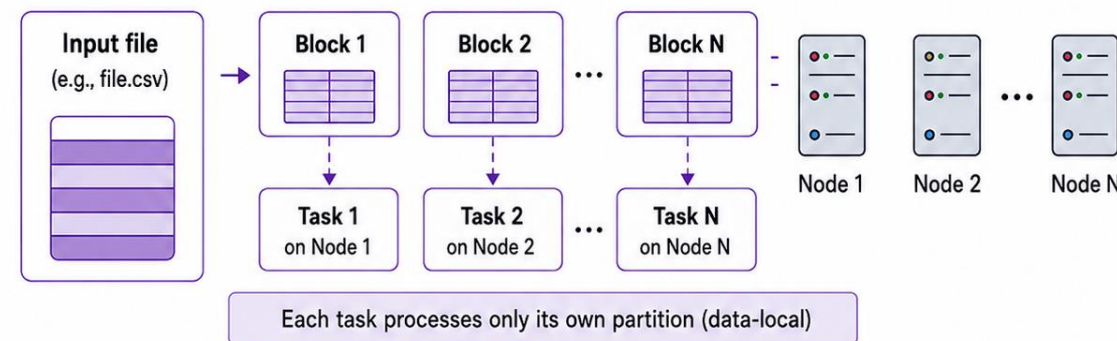
X-AXIS: SCALING OUT (HORIZONTAL SCALING)



Y-AXIS: FUNCTIONAL DECOMPOSITION



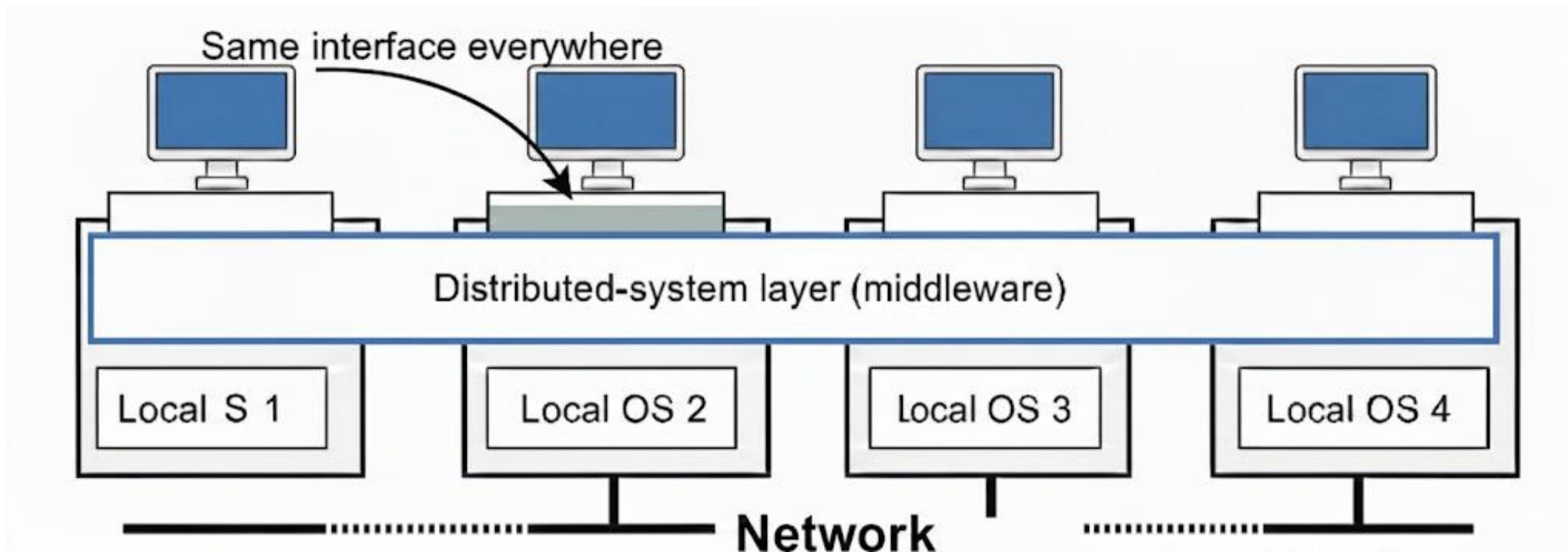
Z-AXIS: DATA PARTITIONING (DATA-LOCAL)



Middleware

L'obiettivo principale del middleware è **nascondere l'eterogeneità** di hardware e sistemi operativi.

I servizi comuni offerti dal middleware includono: **comunicazione** inter-applicativa, **sicurezza**, **logging**, **masking** e **recovery** dei guasti, **naming** e **gestione delle transazioni**.



Le forme della trasparenza

Tipo di Trasparenza	Descrizione
Accesso	Nasconde le differenze nella rappresentazione dei dati e nei meccanismi di accesso .
Posizione (Location)	Nasconde l'esatta ubicazione fisica di una risorsa.
Rilocazione	Nasconde lo spostamento di una risorsa mentre è in uso.
Migrazione	Nasconde lo spostamento di una risorsa a riposo.
Replica	Nasconde l'esistenza di copie multiple della stessa risorsa.
Concorrenza	Nasconde l'uso simultaneo della risorsa da parte di più utenti.
Guasti (Failure)	Nasconde il malfunzionamento e il ripristino di una risorsa.

Hadoop: un po' di storia

2003: Google pubblica la propria **architettura cluster e il file system distribuito (GFS)**

2004: Google pubblica il proprio **modello di programmazione MapReduce**, costruito sopra GFS.

- ↳ Scritto in C++. Closed-source.
- ↳ API in Python e Java.
- ↳ Disponibile solo per i programmatori di Google.

2006: Apache & Yahoo! → Hadoop & HDFS:

- ↳ implementazioni open-source in Java di MapReduce e GFS di Google.
- ↳ un insieme variegato di API disponibili pubblicamente.

2008: diventa un progetto Apache indipendente. Yahoo! usa Hadoop in produzione.

Oggi: utilizzato come **piattaforma general-purpose di archiviazione e analisi** per i big data.

Esistono altre distribuzioni Hadoop fornite da diversi vendor, tra cui EMC, IBM, Microsoft, Oracle, Cloudera, Hortonworks, ecc.

L'architettura (core) di Hadoop

Componenti indipendenti

■ HDFS

- Un file system distribuito.
- Idea: i server possono guastarsi senza interrompere la computazione ⇒ i dati sono replicati con ridondanza all'interno del cluster.

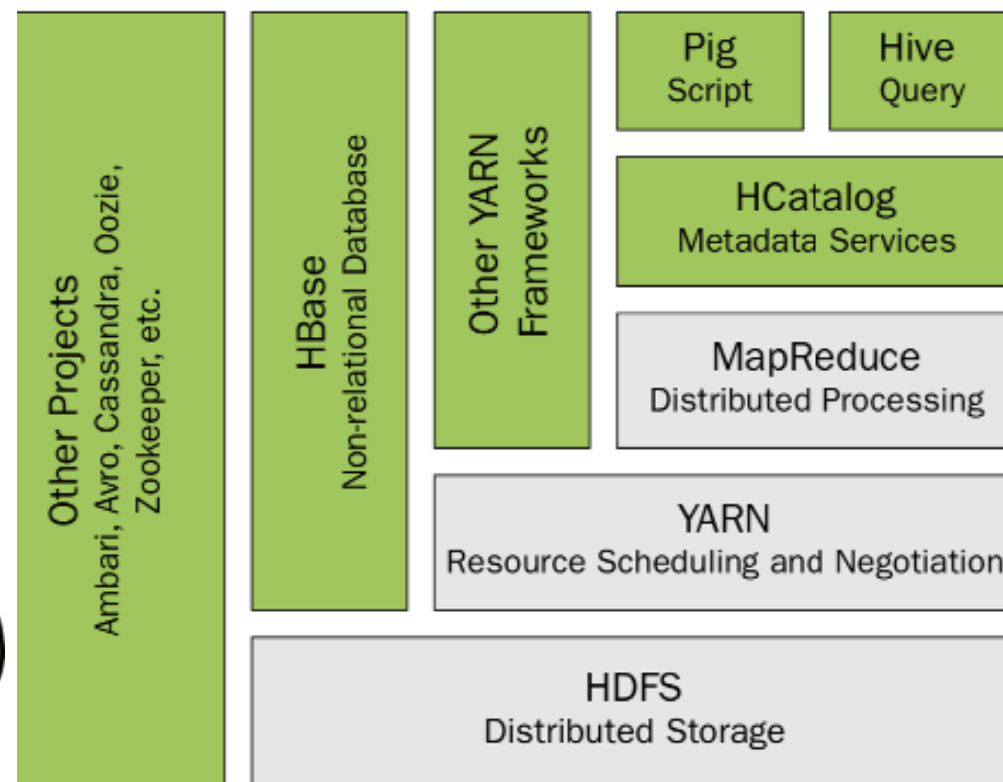
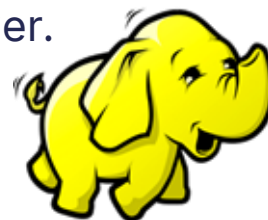
■ YARN

- Gestione delle risorse del cluster.
- Scheduling dei processi.

■ MapReduce

- Paradigma di programmazione (libreria e interfacce) per **esprimere** computazioni distribuite su più server.

Non trattato



Cos'è un file system?

- Dal punto di vista di un sistema operativo, un **dispositivo di archiviazione** è un grande array lineare di **byte**. Qualunque dato è semplicemente una sequenza di 0 e 1 (byte).
- Il file è solo un'unità logica per organizzare i dati (byte). Un'area del disco viene designata come appartenente a un file.
- Le applicazioni (e gli utenti) hanno bisogno di un'astrazione di livello più alto per l'archiviazione
 - Naming basato su stringhe (es. "photos/Livia.jpg")
 - Gestione automatica dei byte liberi e allocati al variare di file creati ed eliminati
- I file hanno tipicamente attributi come:
 1. Nome, Identificatore e User ID
 2. Tipo
 3. Posizione
 4. Dimensione
 5. Permessi di accesso
 6. Timestamp

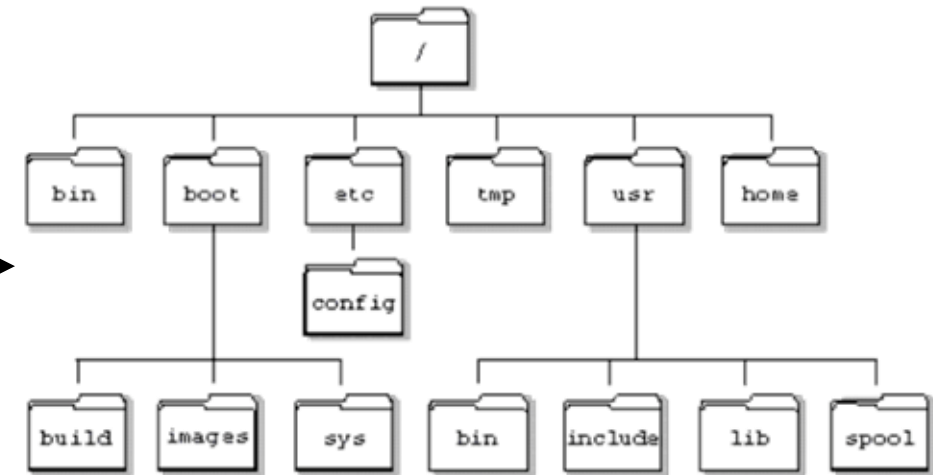


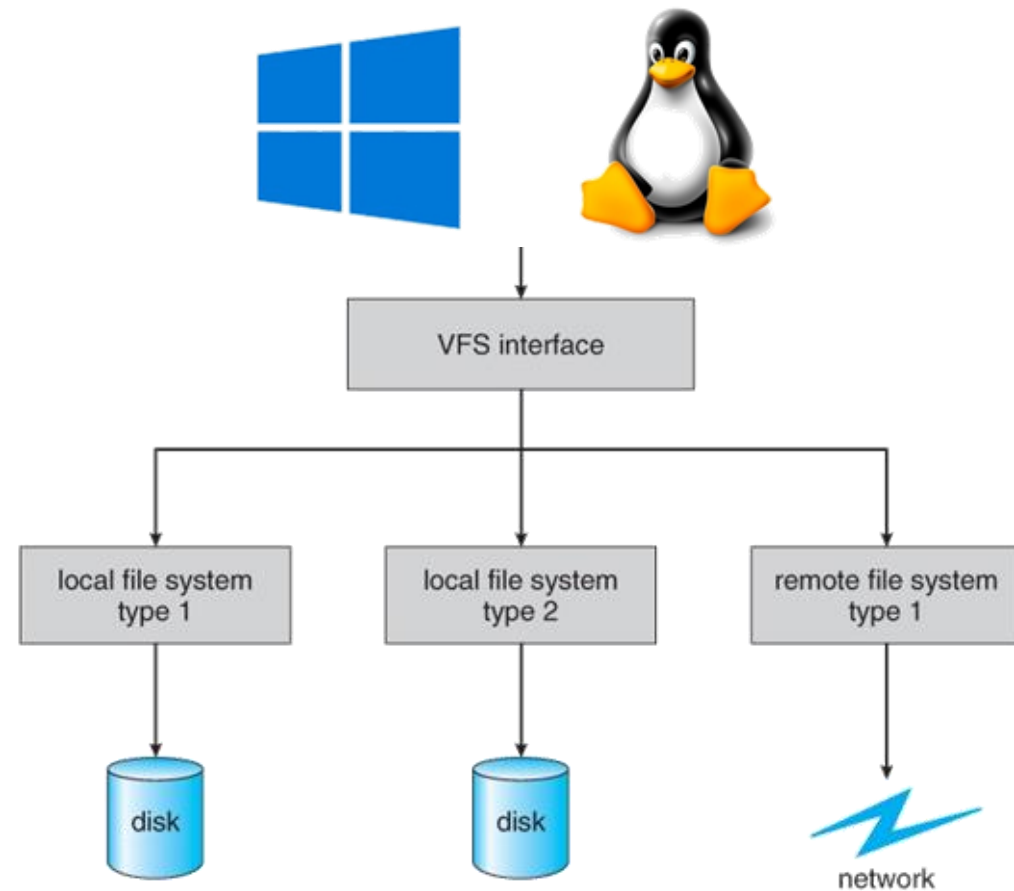
Image Credit: QNX

Sistemi Distribuiti

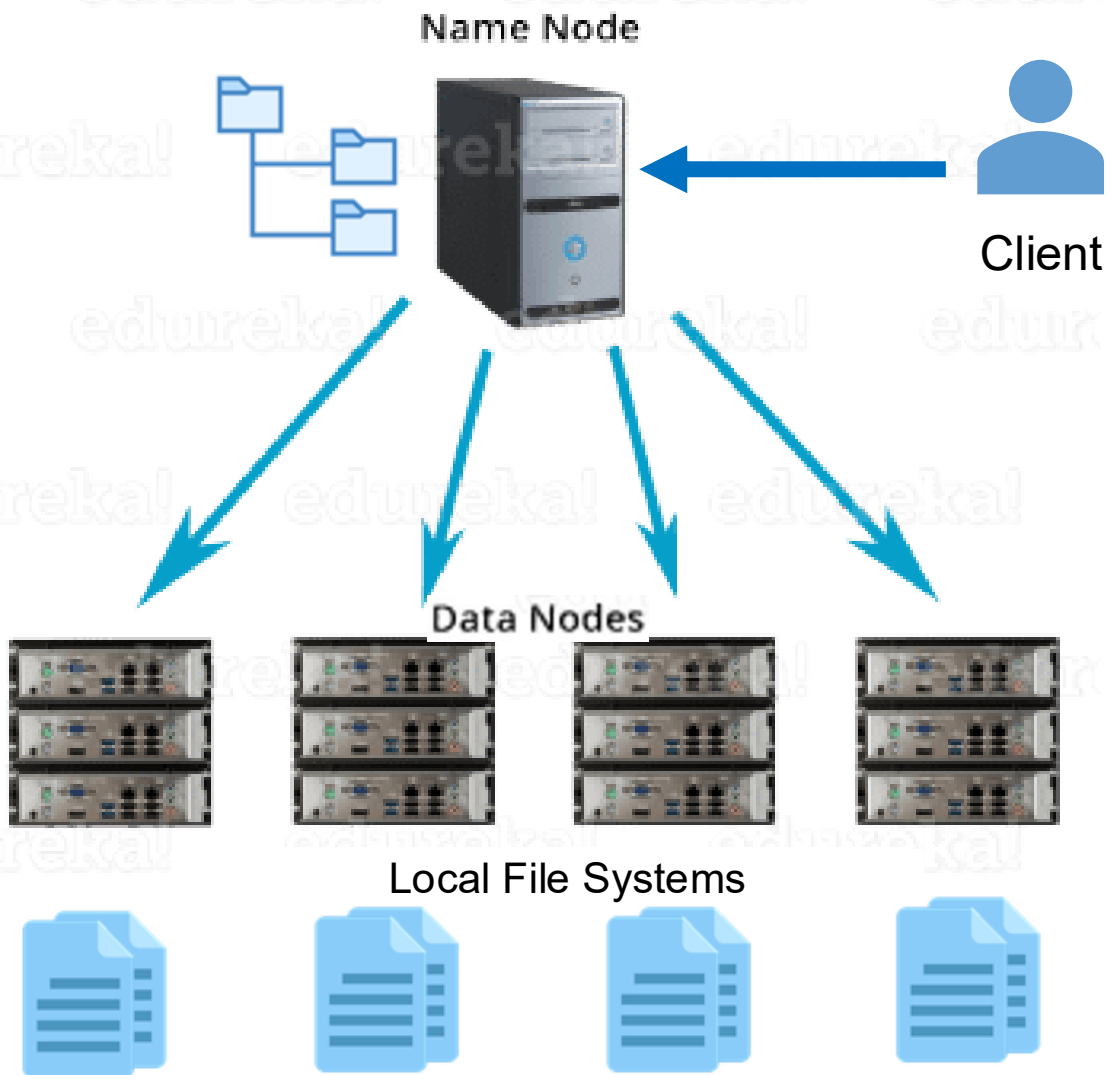
Virtual File System

Un VFS fornisce un'**interfaccia standard** che permette al sistema operativo di interagire con vari file system. Il SO non ha bisogno di conoscere i dettagli interni di funzionamento di ciascun file system.

- In passato, i sistemi operativi potevano usare un solo file system integrato.
- Il **VFS definisce un insieme di metodi** (come open, read, write e close) che ogni file system deve implementare. Questo consente al SO di interagire facilmente con file system diversi tramite un insieme comune di operazioni.
- In sostanza, il nucleo del sistema operativo non ha bisogno di conoscere il funzionamento interno del file system.
- Un VFS permette a un sistema operativo di gestire facilmente uno o più file system contemporaneamente.



Cos'è HDFS?



HDFS è un file system distribuito

- & **fault tolerant**/replicato,
- & **scalabile** e
- & estremamente **facile da espandere**.

Progettato per essere eseguito su **commodity hardware**

- & Le API di HDFS sono implementate anche su soluzioni di Cloud storage

HDFS è lo storage distribuito principale per le applicazioni Hadoop.

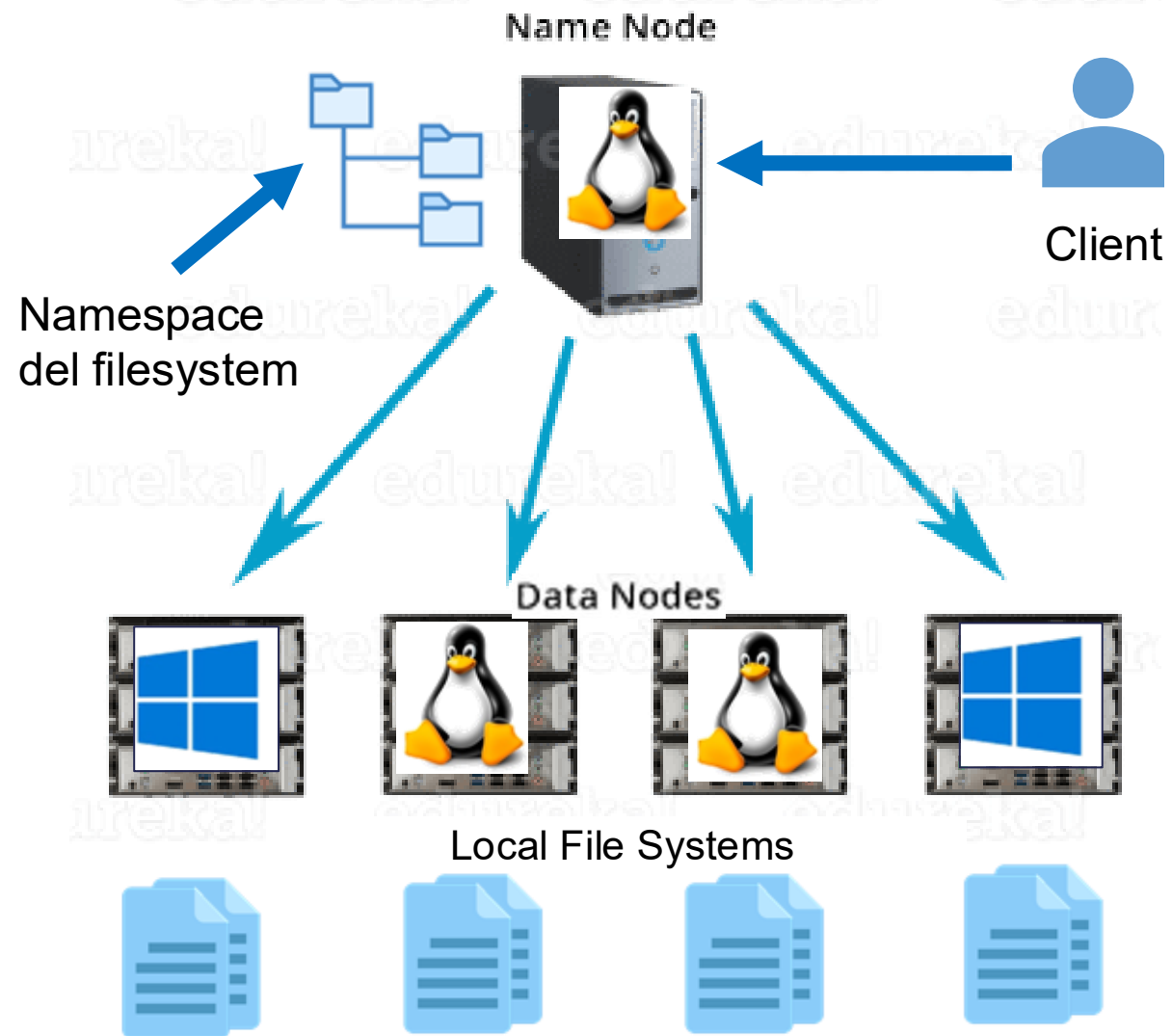
Il passaggio da un Virtual File System (VFS) a un Distributed File System (DFS) rappresenta un'evoluzione nel modo in cui lo storage viene gestito su più macchine.

Mentre un VFS astrae l'interazione tra un sistema operativo e diversi file system locali, un **DFS** porta il concetto oltre, distribuendo i dati su più macchine o server e rendendoli accessibili come se si trovassero su un unico sistema.

Hadoop Distributed File System (HDFS)

Apache HDFS (Hadoop Distributed File System)

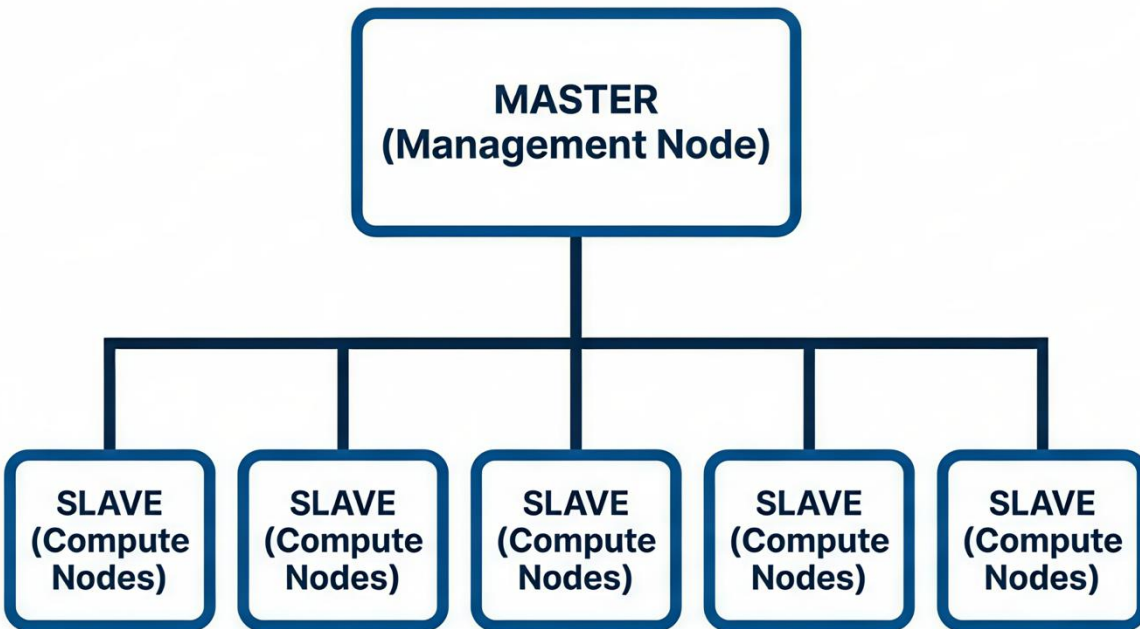
- ❏ **file system a blocchi** (ogni file viene suddiviso in blocchi di dimensione predefinita).
- ❏ i blocchi vengono memorizzati su un **cluster** di macchine.
- ❏ segue l'**architettura Manager/Worker**: un cluster comprende un singolo **NameNode (nodo Manager)** e tutti gli altri nodi sono **DataNode (nodi Worker)**.
- ❏ HDFS può essere installato su un'ampia gamma di macchine che supportano **Java**.



Il **namespace del file system** indica la struttura e l'organizzazione di nomi di file e directory all'interno di un file system. Definisce come i file e le directory vengono nominati, acceduti e rappresentati nel file system, permettendo a utenti e applicazioni di interagire con i dati memorizzati.

Manager-Worker (a.k.a. Master-slave)

L'**architettura manager-worker** è un modello in cui un singolo nodo (o processo) manager *coordina e gestisce* più nodi (o processi) worker che eseguono le effettive attività di calcolo o di elaborazione dei dati. Il nodo manager *assegna i compiti* ai worker e può anche occuparsi dell'aggregazione dei risultati.

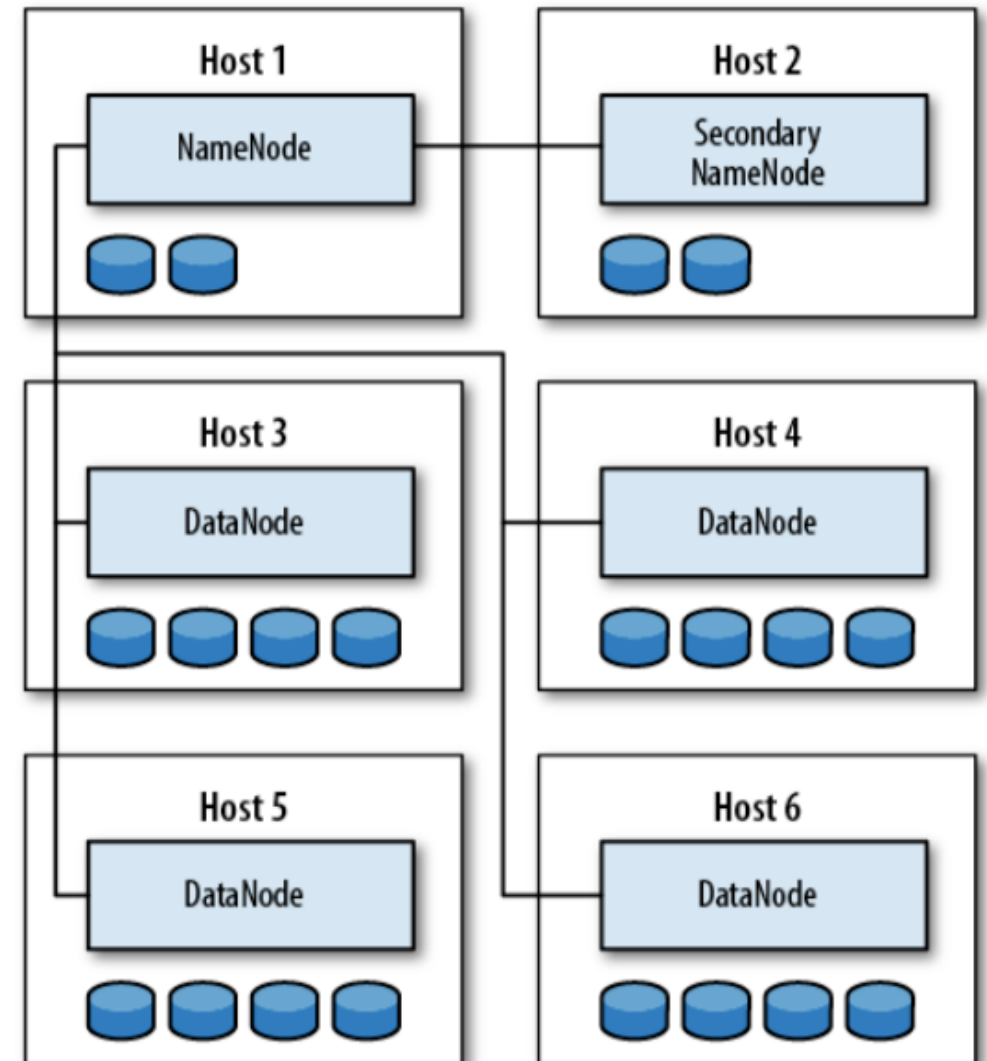


Architettura HDFS

HDFS ha un'**architettura manager-worker**

- Un cluster HDFS è composto da un singolo **NameNode**, un server manager che gestisce il namespace del file system e regola l'accesso ai file da parte dei client
- Sono presenti più **DataNode**, che gestiscono i volumi di archiviazione collegati ai nodi su cui sono in esecuzione

HDFS espone **un unico file system** per l'intero cluster e permette di memorizzare i dati utente in file all'interno dei DataNode



Architettura HDFS

Internamente, un file viene suddiviso in uno o più **blocchi** (detti anche chunk) e questi blocchi vengono memorizzati in un insieme di **DataNode**

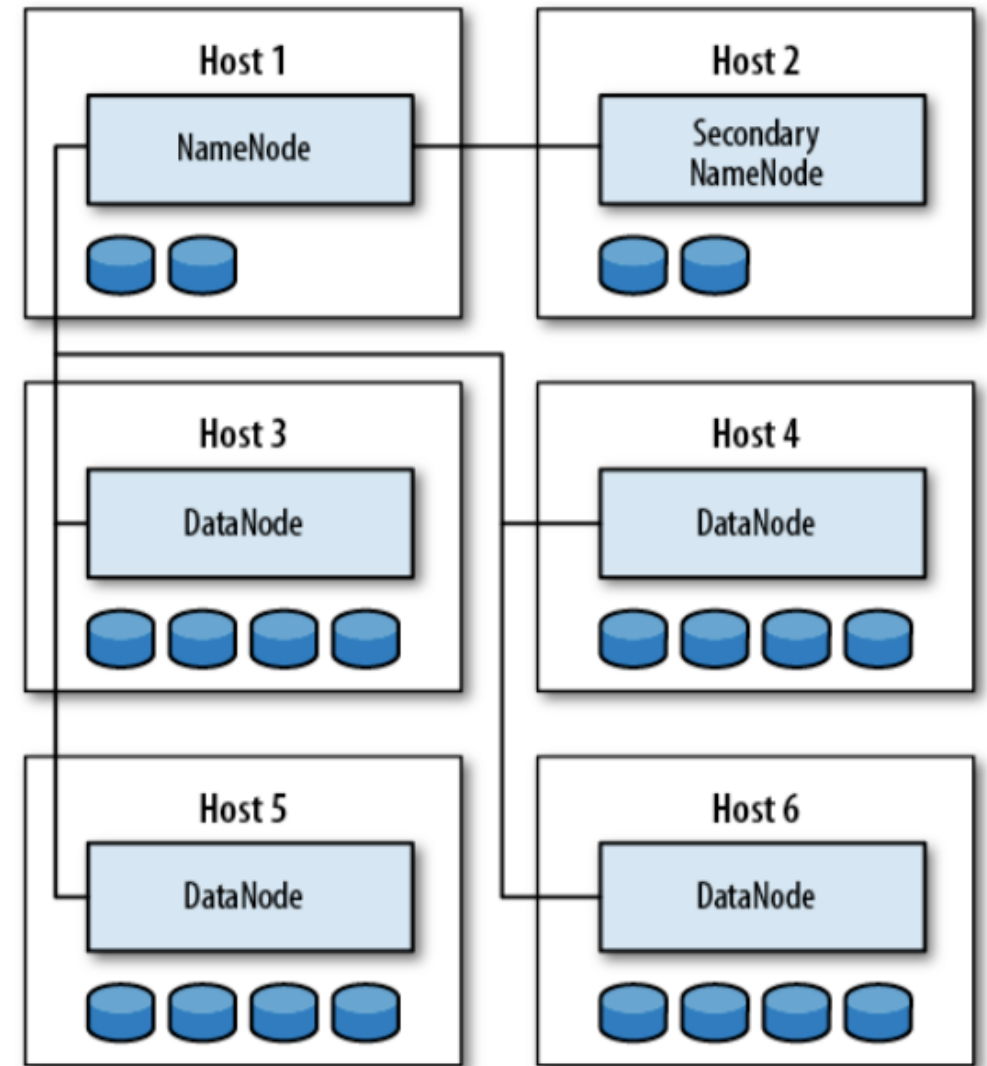
⌘ I chunk vengono **replicati** (per default 3 volte)

Il NameNode:

- ⌘ esegue le operazioni sul file system come **apertura**, **chiusura** e **rinomina** di file e directory
- ⌘ determina la mappatura dei blocchi sui DataNode

I DataNode sono responsabili di:

- ⌘ **servire le richieste di lettura e scrittura** dei client
- ⌘ **creazione, cancellazione e replica** dei blocchi su istruzione del NameNode



Responsabilità del NameNode (in dettaglio)

Gestione del file system:

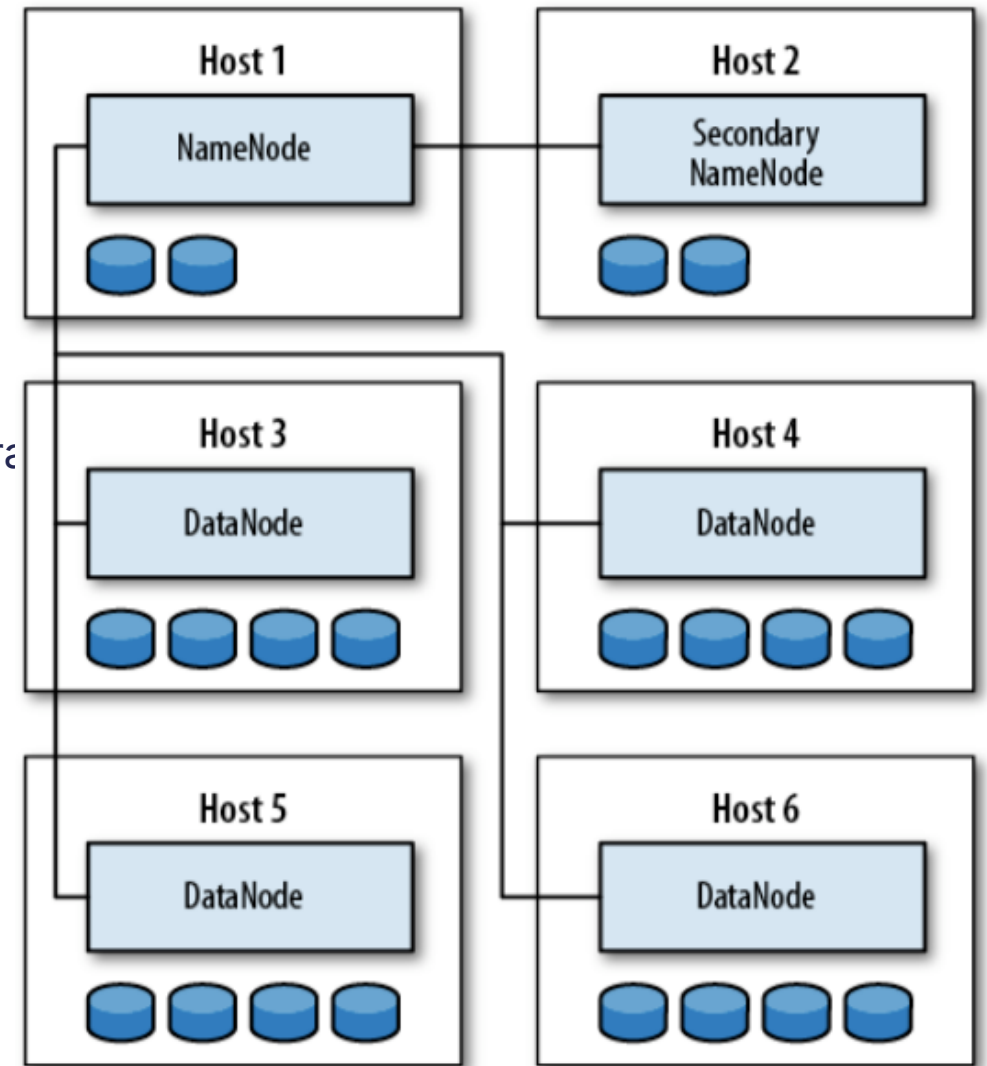
- ⌘ Mantiene i metadati della struttura di file e directory, la mappatura file-blocco, i permessi di accesso, ecc.

Coordinamento delle operazioni sui file:

- ⌘ Indirizza i client verso i DataNode per le operazioni di lettura/scrittura
- ⌘ Infatti, *nessun dato transita attraverso il NameNode*

Gestione dello stato di salute complessivo:

- ⌘ Comunicazione periodica con i DataNode (health check)
- ⌘ Replica e ribilanciamento dei blocchi



Responsabilità del DataNode

Block Server

- Memorizza i dati nel file system **locale** (es. ext3)
- Memorizza i metadati di un blocco (es. CRC)
- Fornisce dati e metadati alle applicazioni client

Block Report

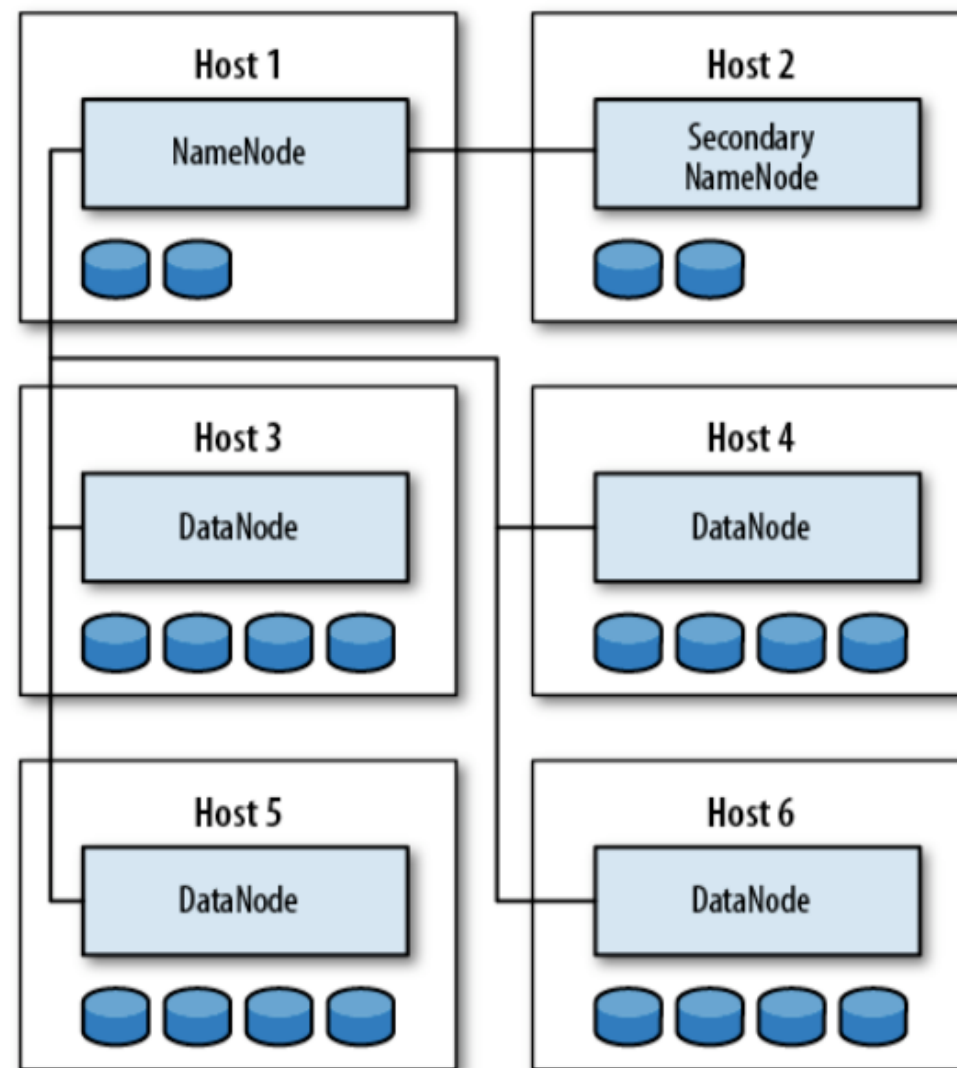
- Periodicamente **invia un report** di tutti i blocchi esistenti al NameNode

Creazione del file

- Il client calcola il checksum (CRC32)
- Il DataNode memorizza il checksum

Accesso al file

- Il client recupera i dati e il checksum dal DataNode
- Se la validazione fallisce, il client tenta con altre repliche



Correttezza dei dati

Replica dei dati in HDFS: scopo e architettura

HDFS è progettato per memorizzare in modo affidabile file di dimensioni molto grandi distribuiti su macchine in un grande cluster

- Memorizza ogni file come una **sequenza di blocchi**
- I blocchi di un file vengono **replicati** per garantire **fault tolerance e prestazioni**

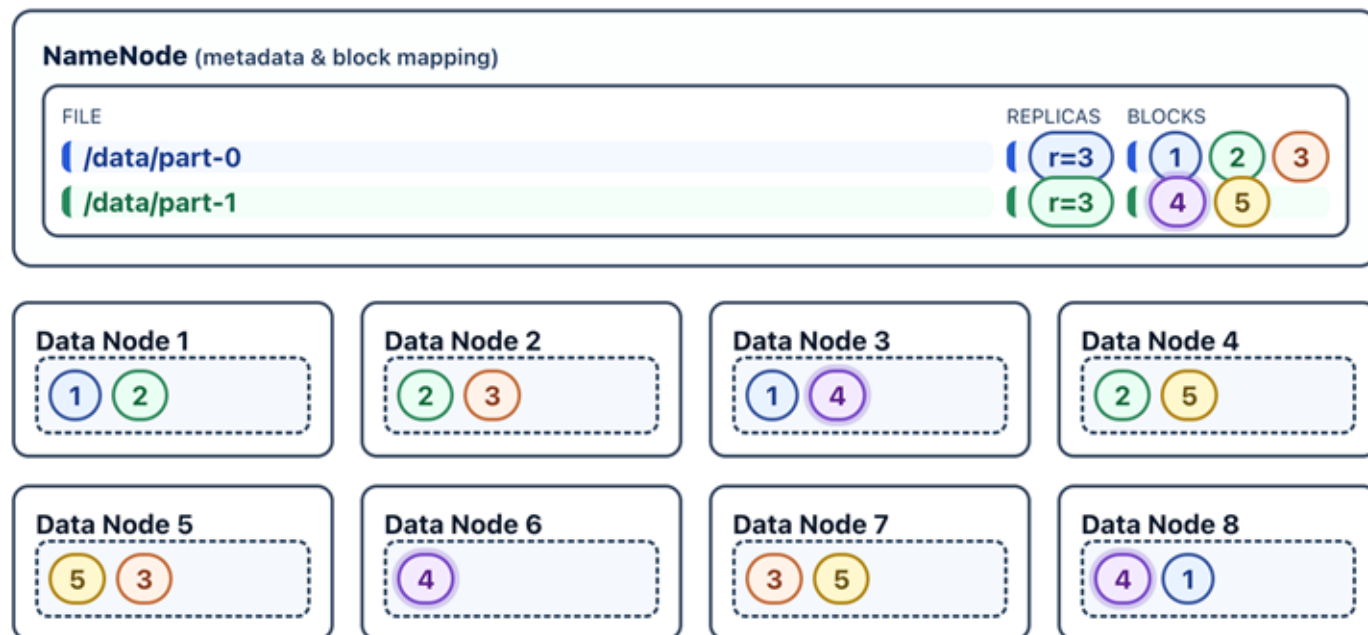
Bisogna aspettarsi i guasti:

- con un solo server, attendersi un guasto ogni 3 anni
- con **migliaia** di server, attendersi **un guasto ogni 10 minuti**

La dimensione del blocco (default 128 MB) e il fattore di replica (3) sono configurabili per ogni singolo file.

- Tutti i blocchi di un file, tranne l'ultimo, hanno la stessa dimensione
- Il **fattore di replica** può essere specificato al momento della creazione del file e modificato successivamente (**ribilanciamento**)

HDFS Block Replication

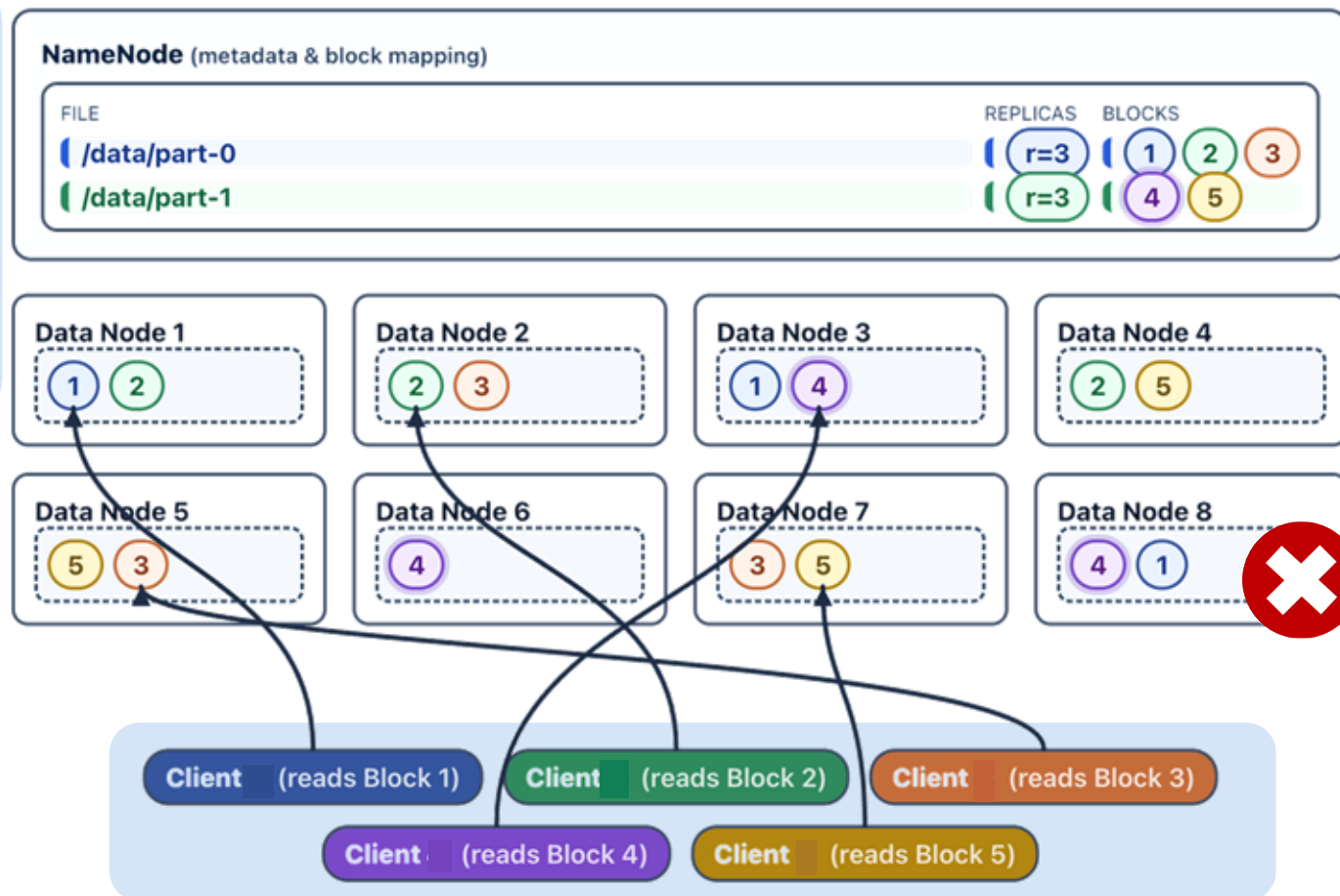


Replica dei dati in HDFS: scopo e architettura

HDFS è progettato per memorizzare in modo affidabile file di dimensioni molto grandi distribuiti su macchine in un grande cluster

- Memorizza ogni file come una **sequenza di blocchi**
- I blocchi di un file vengono **replicati** per garantire **fault tolerance e prestazioni**

HDFS Block Replication



HDFS e le forme della trasparenza

Trasparenza	Implementata	Come in HDFS
Accesso	Sì	API uniforme (FileSystem, hdfs://): l'utente legge/scrive file ignorando rete e formato dei blocchi sottostanti.
Posizione	Sì	Path logici (/user/...); il NameNode mappa ogni file ai DataNode che ospitano i blocchi.
Rilocazione	No	HDFS è write-once: i blocchi di un file aperto non vengono spostati durante l'uso.
Migrazione	Parziale	Il Balancer ridistribuisce i blocchi tra DataNode a riposo, in modo trasparente al client.
Replica	Sì	Ogni blocco replicato (default 3×) su nodi/rack diversi; il client vede un solo file logico.
Concorrenza	Limitata	Single-writer/multi-reader: più letture parallele, una sola scrittura per volta (lease del NameNode).
Guasti	Sì	Heartbeat + ri-replicazione automatica dei blocchi; HA NameNode con standby.

L'architettura (core) di Hadoop

Componenti indipendenti

■ HDFS

- Un file system distribuito.
- Idea: i server possono guastarsi senza interrompere la computazione ⇒ i dati sono replicati con ridondanza all'interno del cluster.

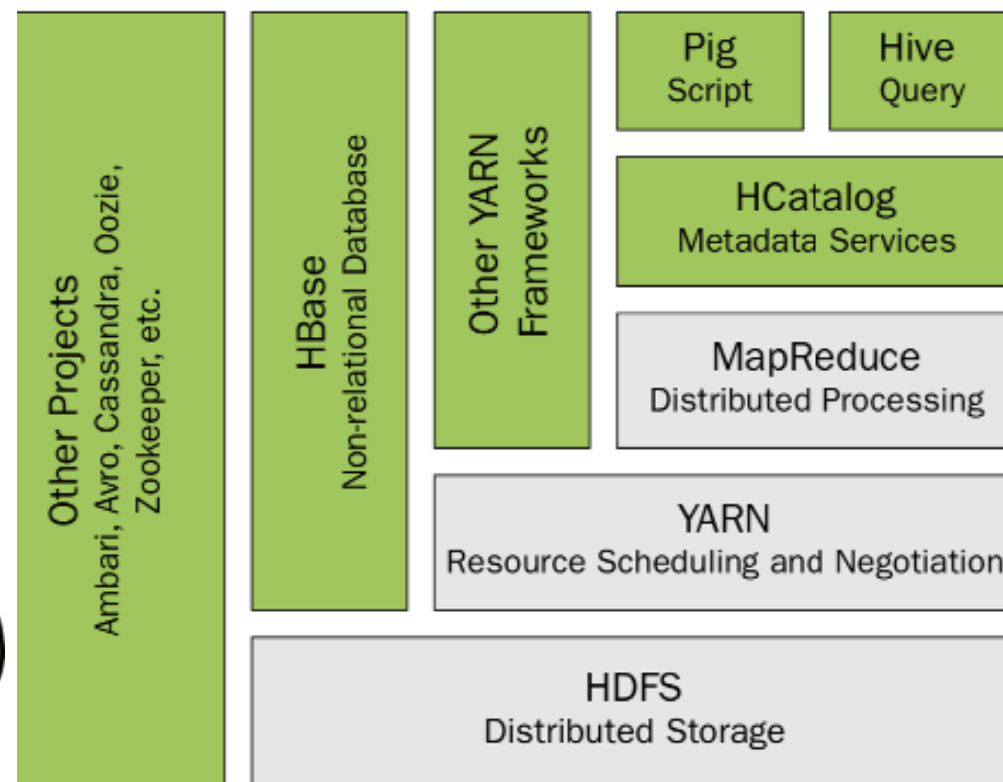
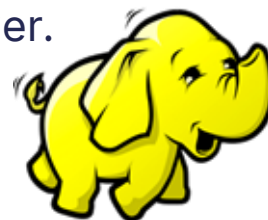
■ YARN

- Gestione delle risorse del cluster.
- Scheduling dei processi.

■ MapReduce

- Paradigma di programmazione (libreria e interfacce) per **esprimere** computazioni distribuite su più server.

Non trattato



Perché YARN?

Apache YARN è un software che gira su un gruppo di macchine e le condivide in modo equo tra molti programmi. Decide quanta potenza di calcolo assegnare a ciascun programma, avvia e monitora i programmi durante l'esecuzione e impedisce che un programma rallenti o blocchi gli altri. YARN può eseguire molti tipi di workload — job batch, streaming, servizi e machine learning — non solo "big data".

Separazione delle responsabilità:

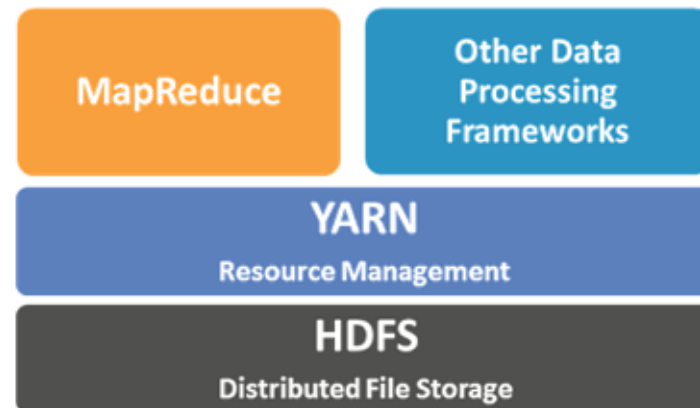
- Gestione delle risorse del cluster
- Aperto a diversi modelli di elaborazione

YARN: Yet Another Resource Negotiator

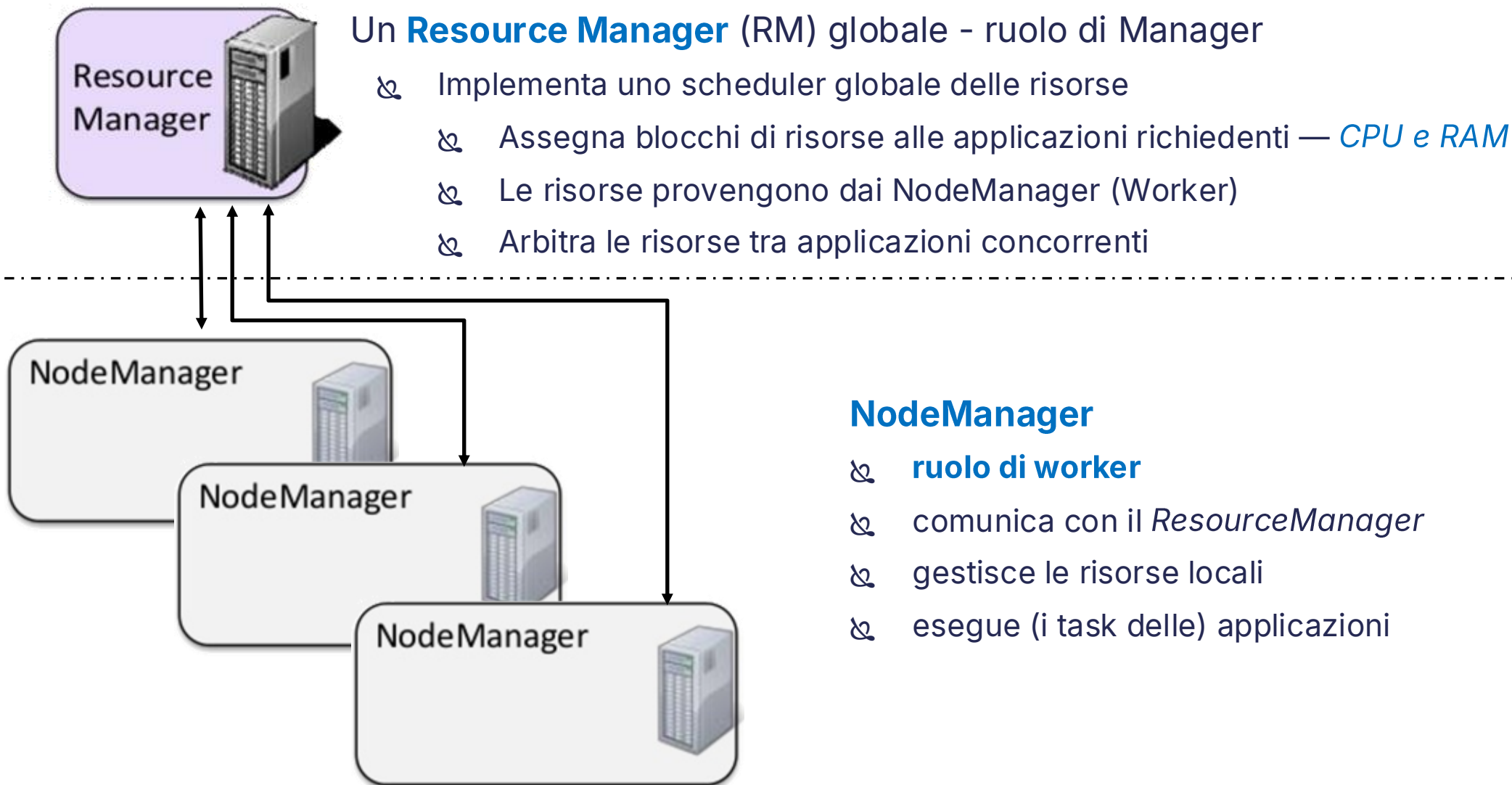
- MapReduce è solo uno dei possibili modelli/framework di elaborazione

YARN è un motore che nasconde i complessi dettagli della *gestione a runtime*

- **parallelizzazione**
- **load balancing (bilanciamento del carico)**
- **ottimizzazione dei trasferimenti su disco**
- **gestione dei guasti delle macchine**

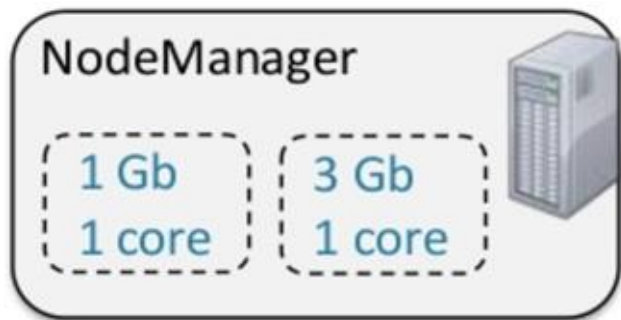


Architettura YARN: ancora manager-worker



Container YARN

Disclaimer: i container YARN sono diversi dai container Docker/Linux. Non è coinvolta alcuna virtualizzazione: i container YARN rappresentano l'assegnazione di risorse ai task.



Container: rappresentazione logica delle risorse (CPU e RAM)

- creati **su richiesta del ResourceManager**
- riservano una certa quantità di risorse su un nodo
- le applicazioni vengono eseguite in uno o più container

Quando viene sottomessa un'applicazione YARN, viene avviato un **Application Master** che ne supervisiona l'esecuzione. Negozia le risorse con il YARN Resource Manager e monitora l'esecuzione dei **task**.

Un **Application Master** (AM) per ogni applicazione compatibile con YARN

- uno per applicazione
- viene eseguito in un container
- richiede ulteriori container per eseguire i **task**

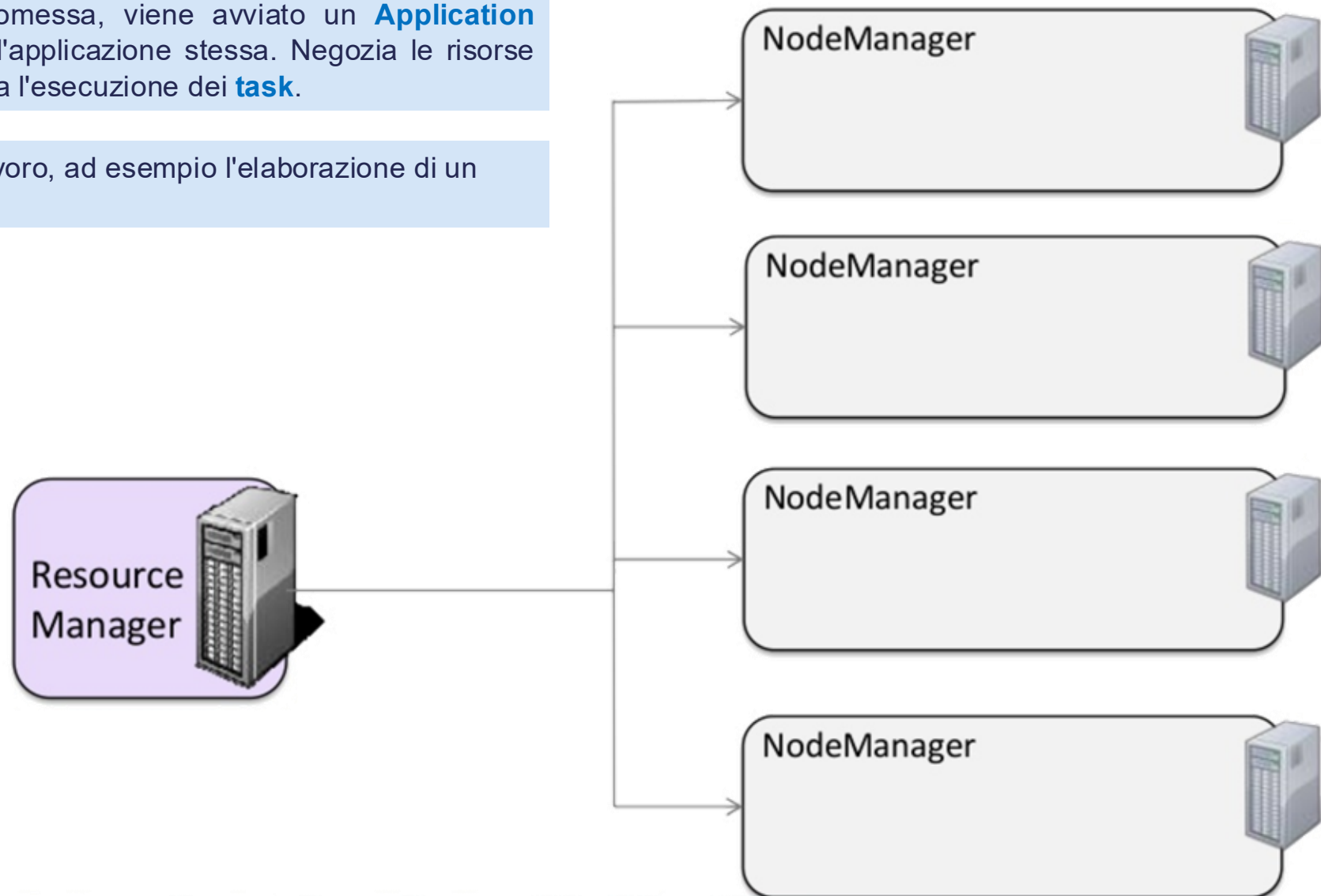
Un **task** svolge una porzione specifica di lavoro, come l'elaborazione di un sottoinsieme di dati in un job MapReduce.



Esecuzione di un'applicazione in YARN

Quando un'applicazione YARN viene sottomessa, viene avviato un **Application Master** per supervisionare l'esecuzione dell'applicazione stessa. Negozia le risorse con il Resource Manager di YARN e monitora l'esecuzione dei **task**.

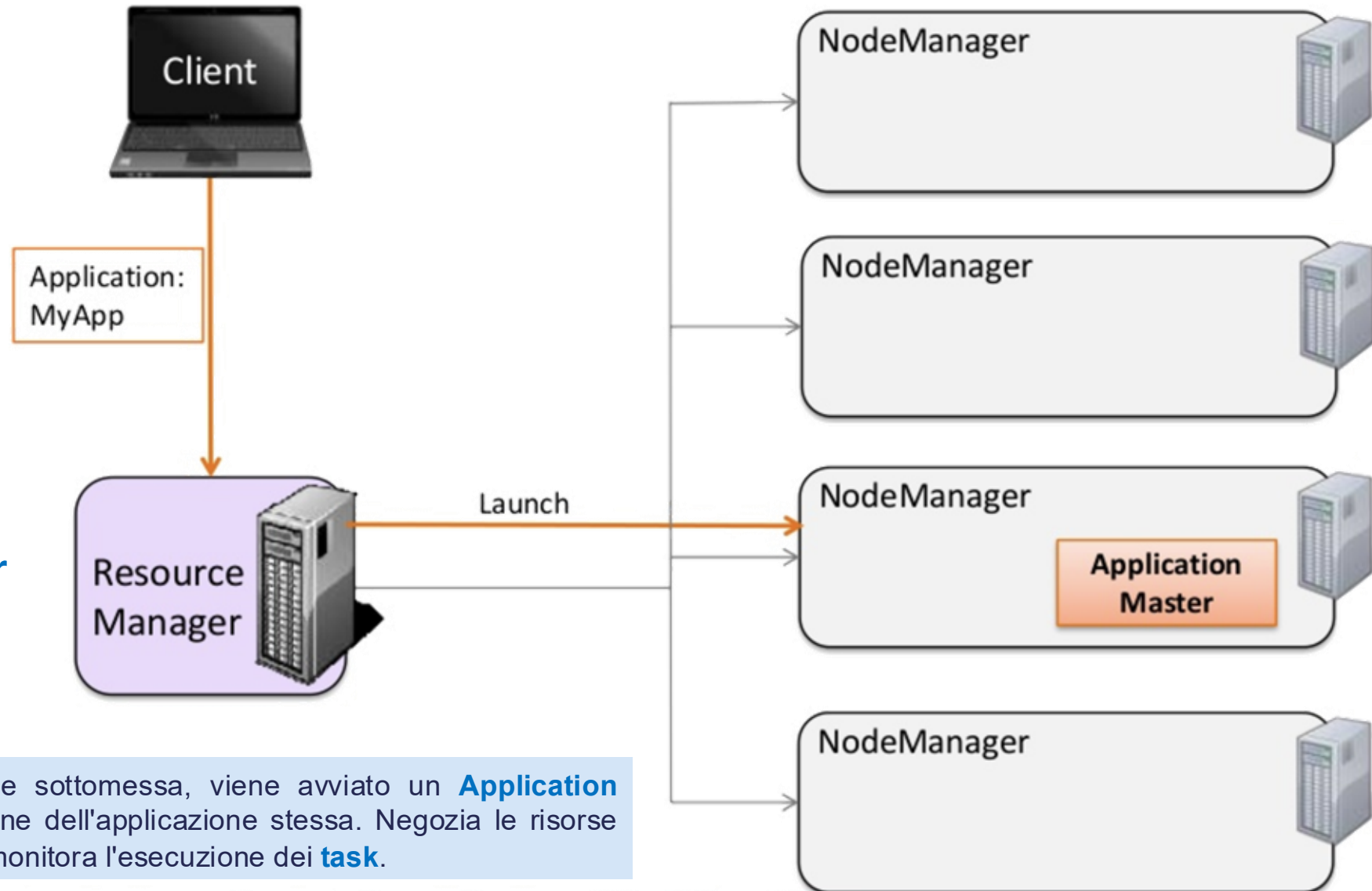
Un **task** esegue una porzione specifica di lavoro, ad esempio l'elaborazione di un sottoinsieme di dati in un job MapReduce.



Esecuzione di un'applicazione in YARN

Il client sottomette un'applicazione compatibile con YARN

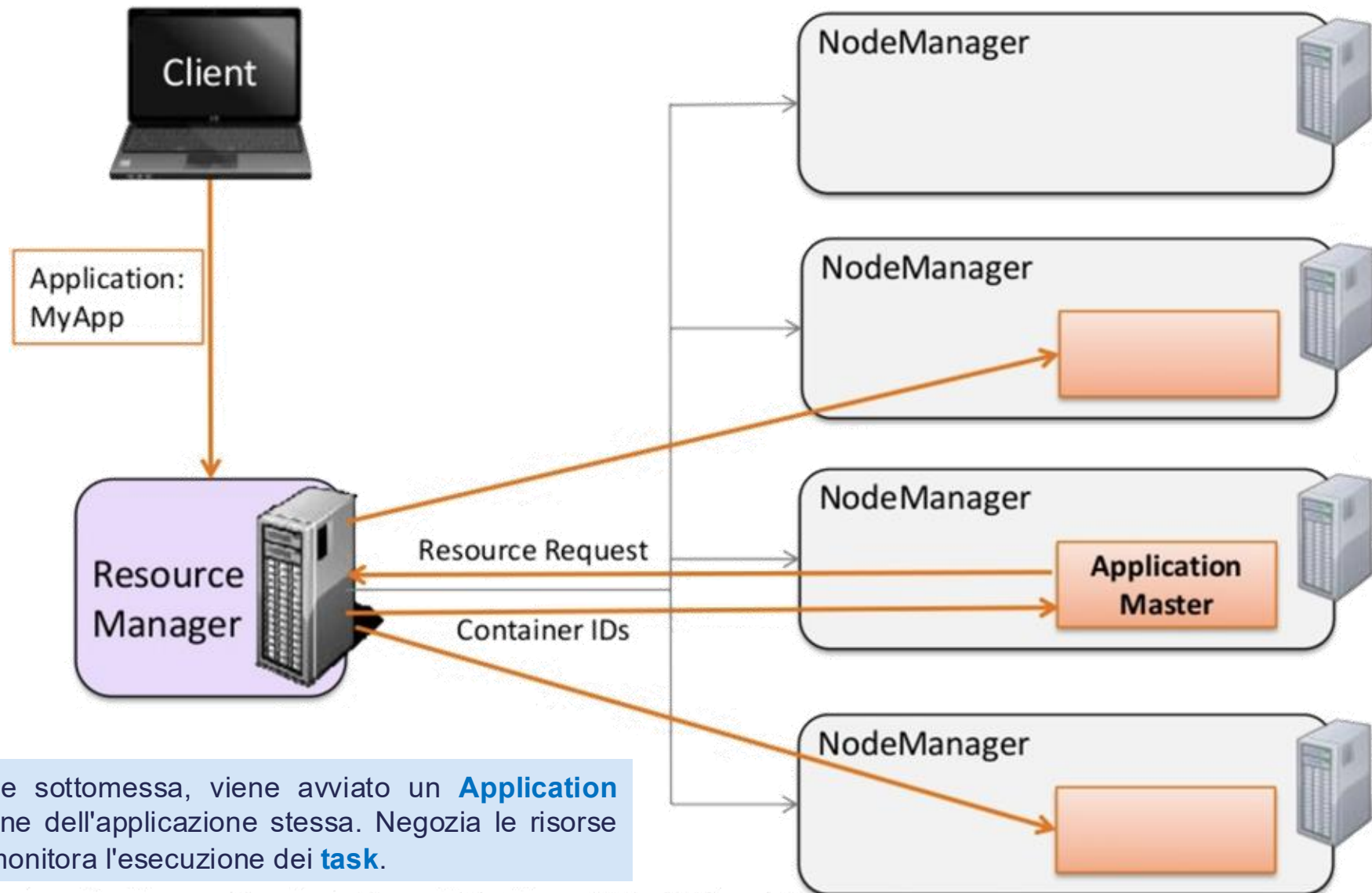
Il RM alloca un container e avvia l'**Application Master**



Quando un'applicazione YARN viene sottomessa, viene avviato un **Application Master** per supervisionare l'esecuzione dell'applicazione stessa. Negozia le risorse con il Resource Manager di YARN e monitora l'esecuzione dei **task**.

Esecuzione di un'applicazione in YARN

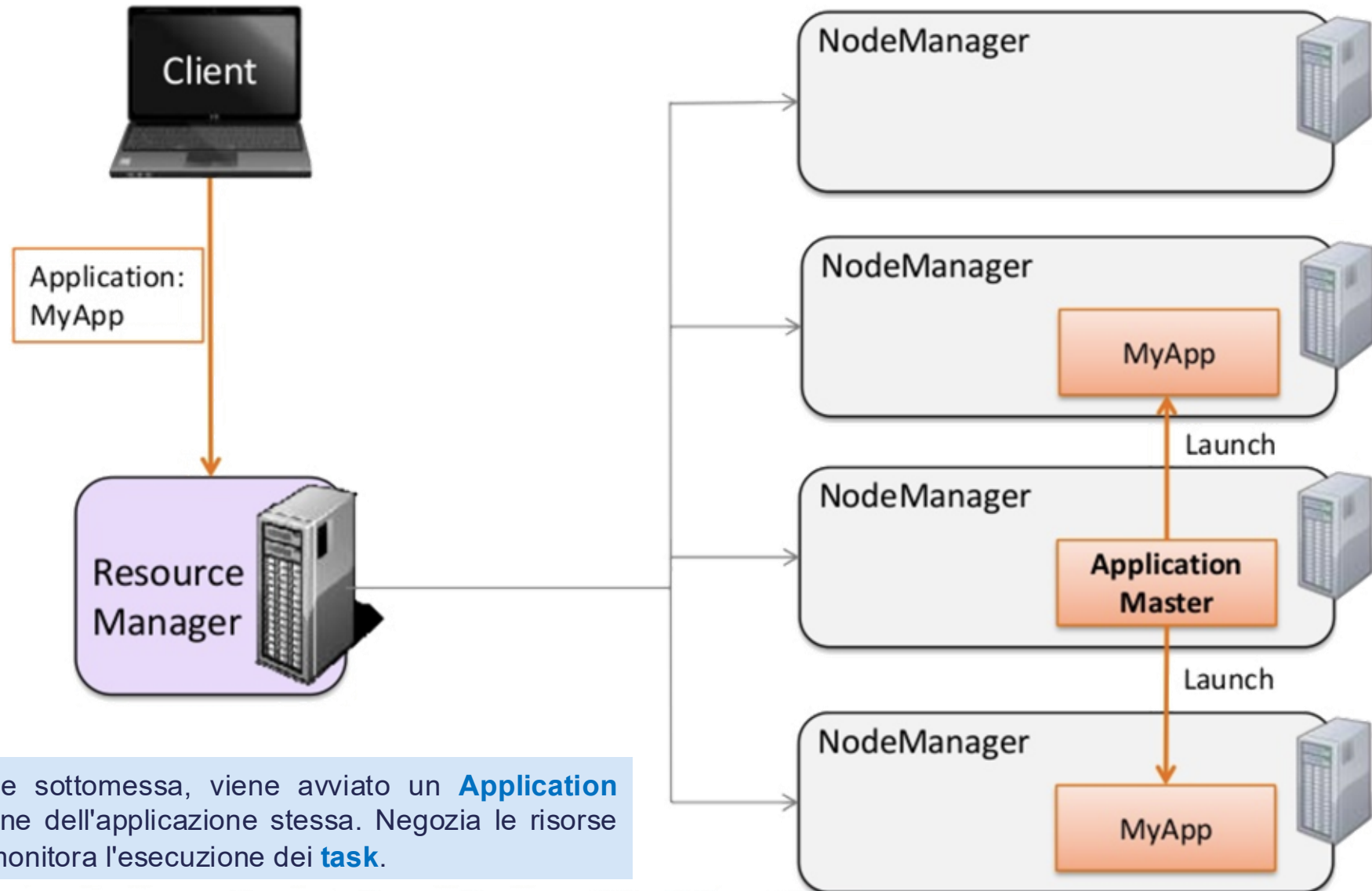
L'**Application Master** richiede i container al Resource Manager



Quando un'applicazione YARN viene sottomessa, viene avviato un **Application Master** per supervisionare l'esecuzione dell'applicazione stessa. Negozia le risorse con il Resource Manager di YARN e monitora l'esecuzione dei **task**.

Esecuzione di un'applicazione in YARN

L'**Application Master** schedula i **task** dell'applicazione sui container.

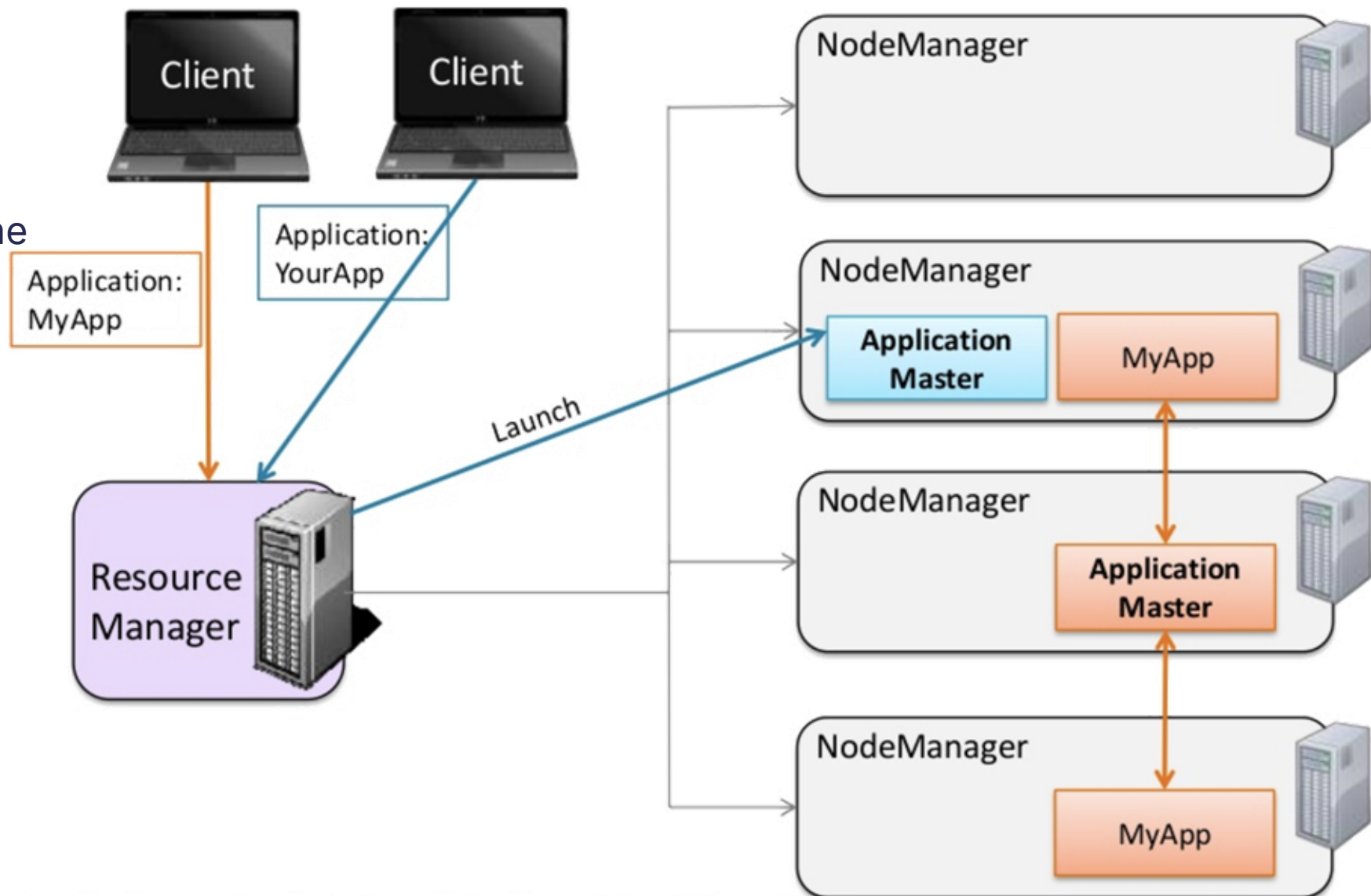


Quando un'applicazione YARN viene sottomessa, viene avviato un **Application Master** per supervisionare l'esecuzione dell'applicazione stessa. Negozia le risorse con il Resource Manager di YARN e monitora l'esecuzione dei **task**.

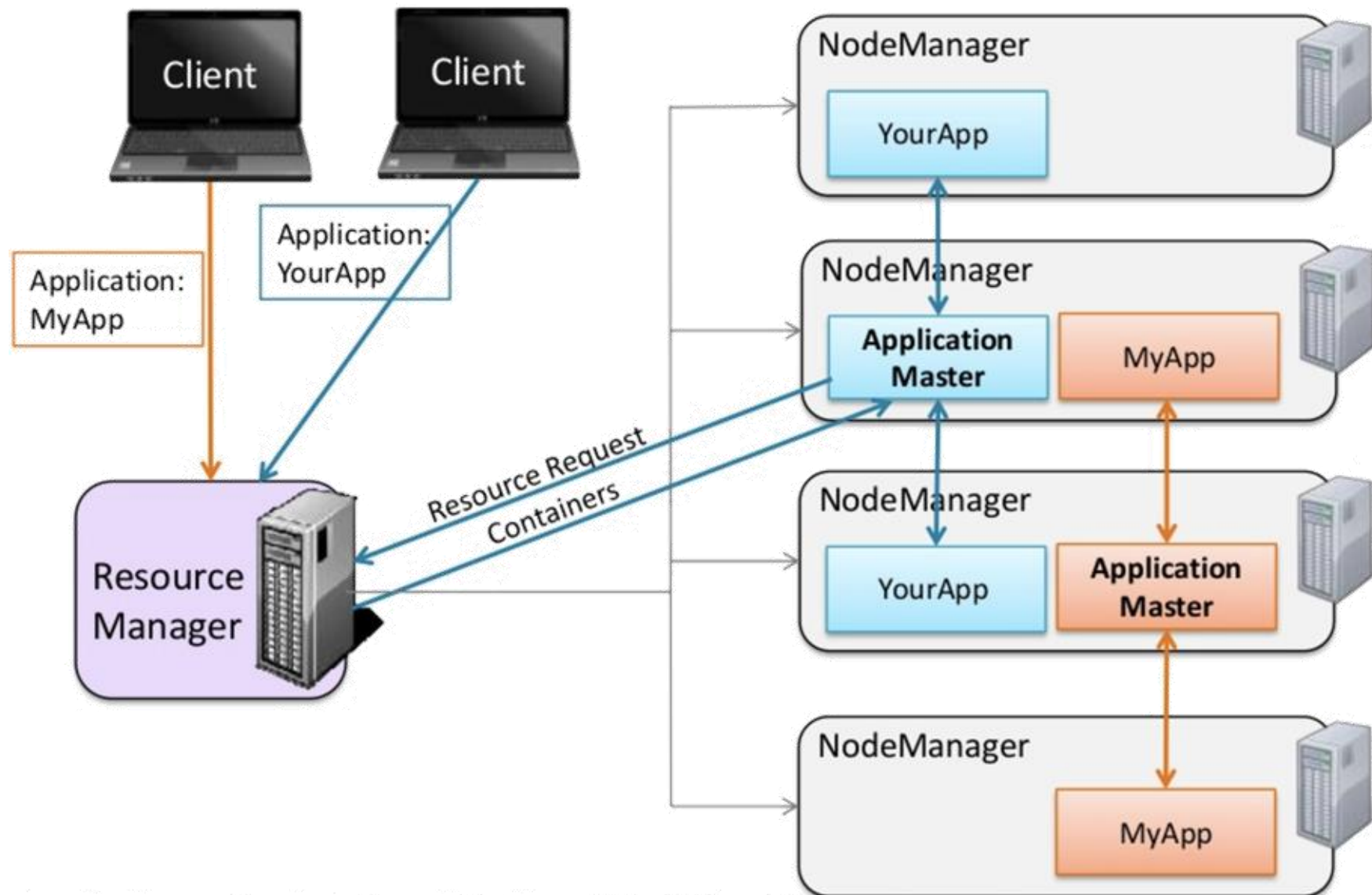
Esecuzione di un'applicazione in YARN

Il cluster è multi-tenant

Il **Resource Manager** può gestire più applicazioni grazie alle proprie politiche di allocazione



Esecuzione di un'applicazione in YARN

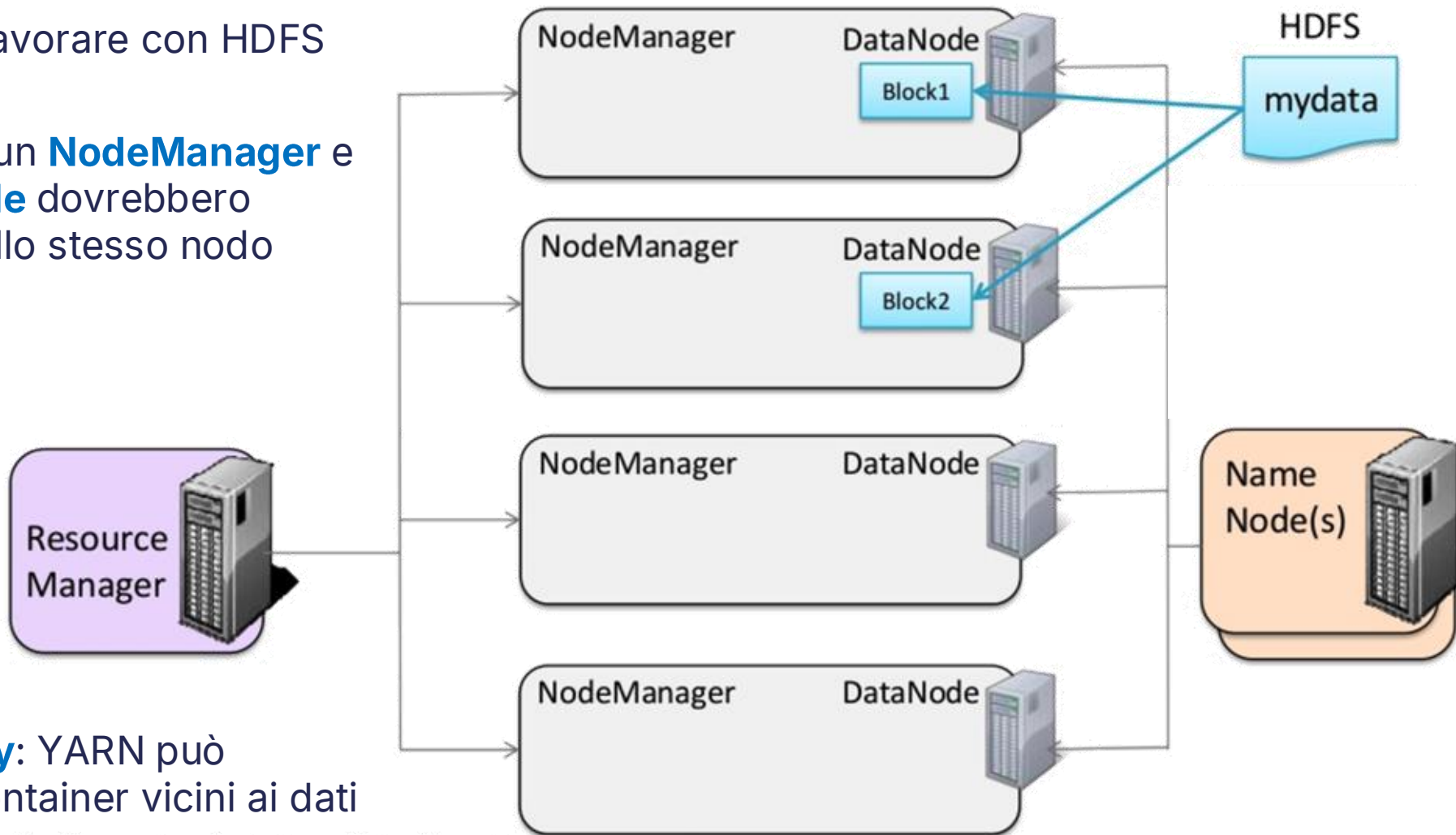


Data locality in YARN+HDFS

Quando un'applicazione YARN (per esempio creata con la libreria MapReduce) viene sottomessa, viene avviato un **Application Master** per supervisionare l'esecuzione dell'applicazione stessa. Negozia le risorse con il Resource Manager di YARN e monitora l'esecuzione dei **task**.

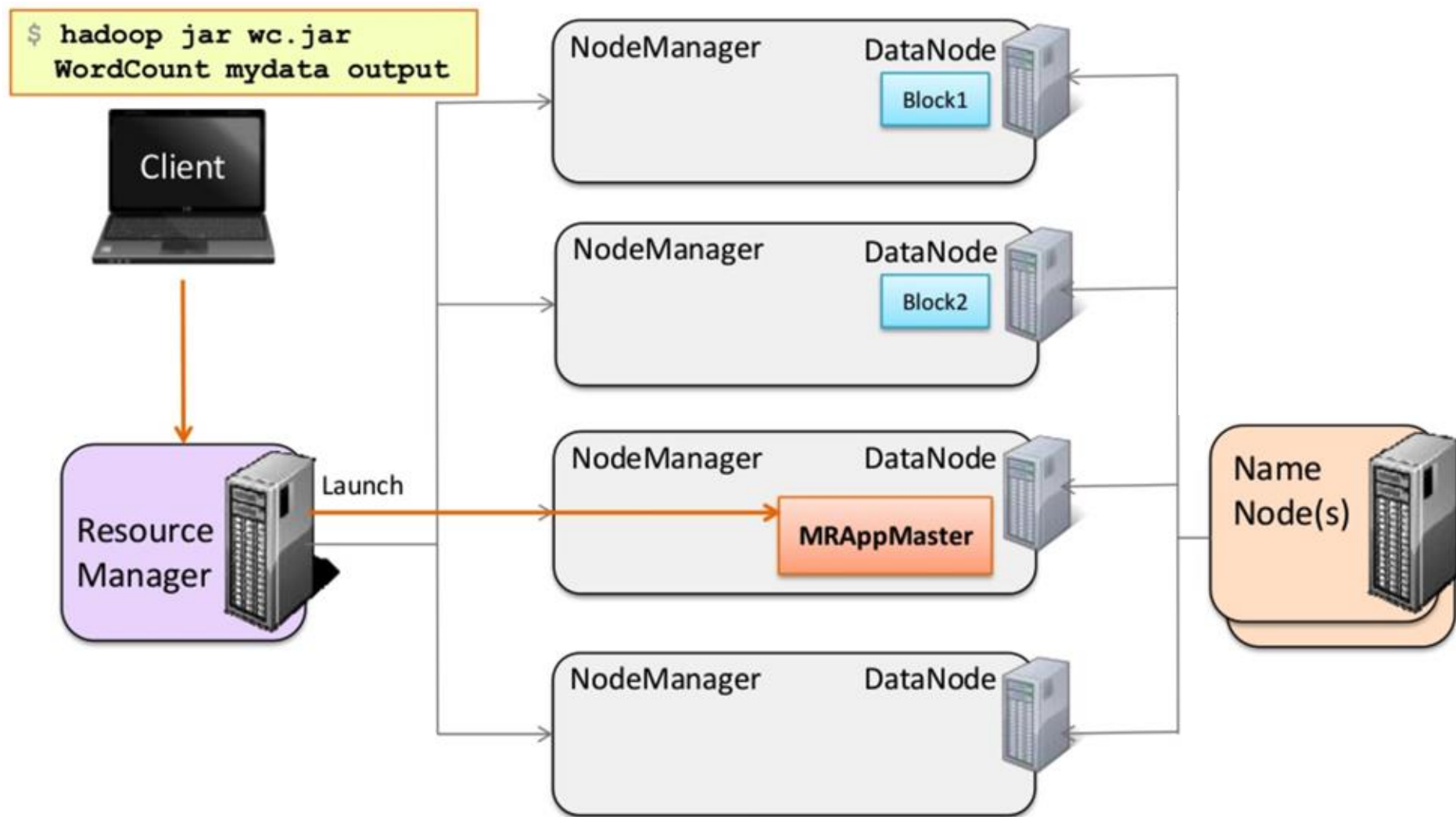
YARN può lavorare con HDFS

Notate che un **NodeManager** e un **DataNode** dovrebbero risiedere sullo stesso nodo

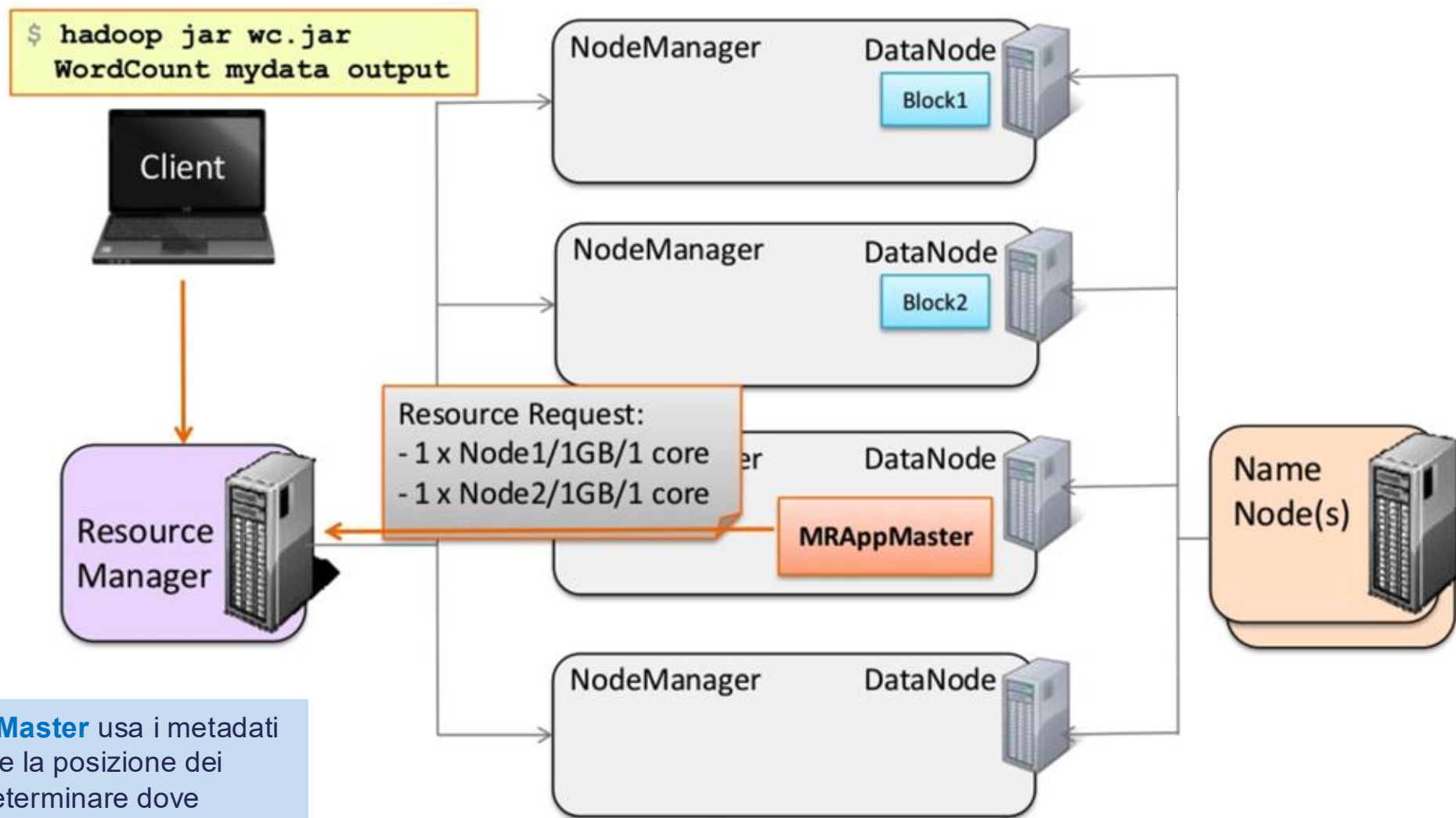


Data locality: YARN può allocare i container vicini ai dati

Data locality in YARN+HDFS

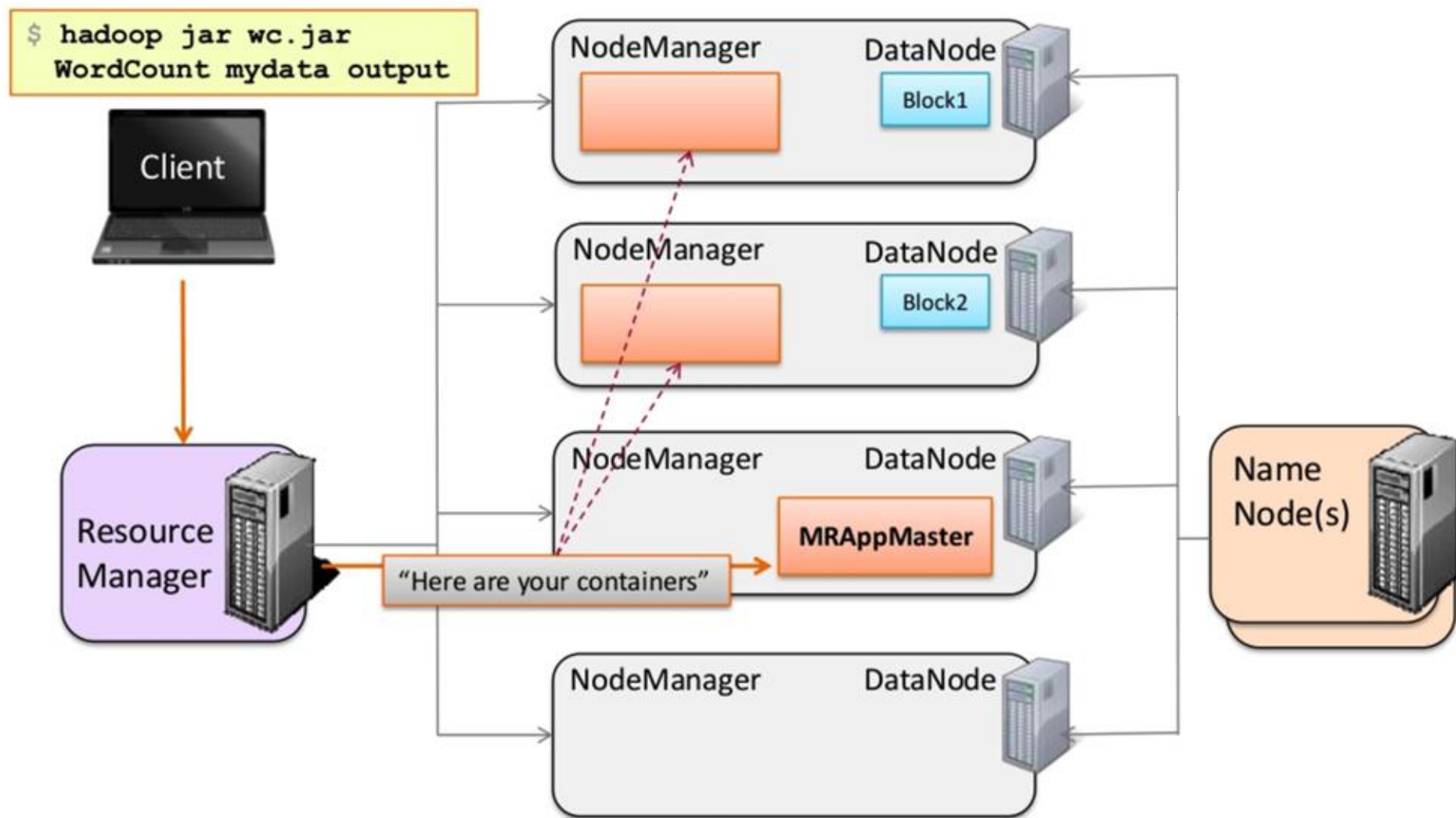


Data locality in YARN+HDFS

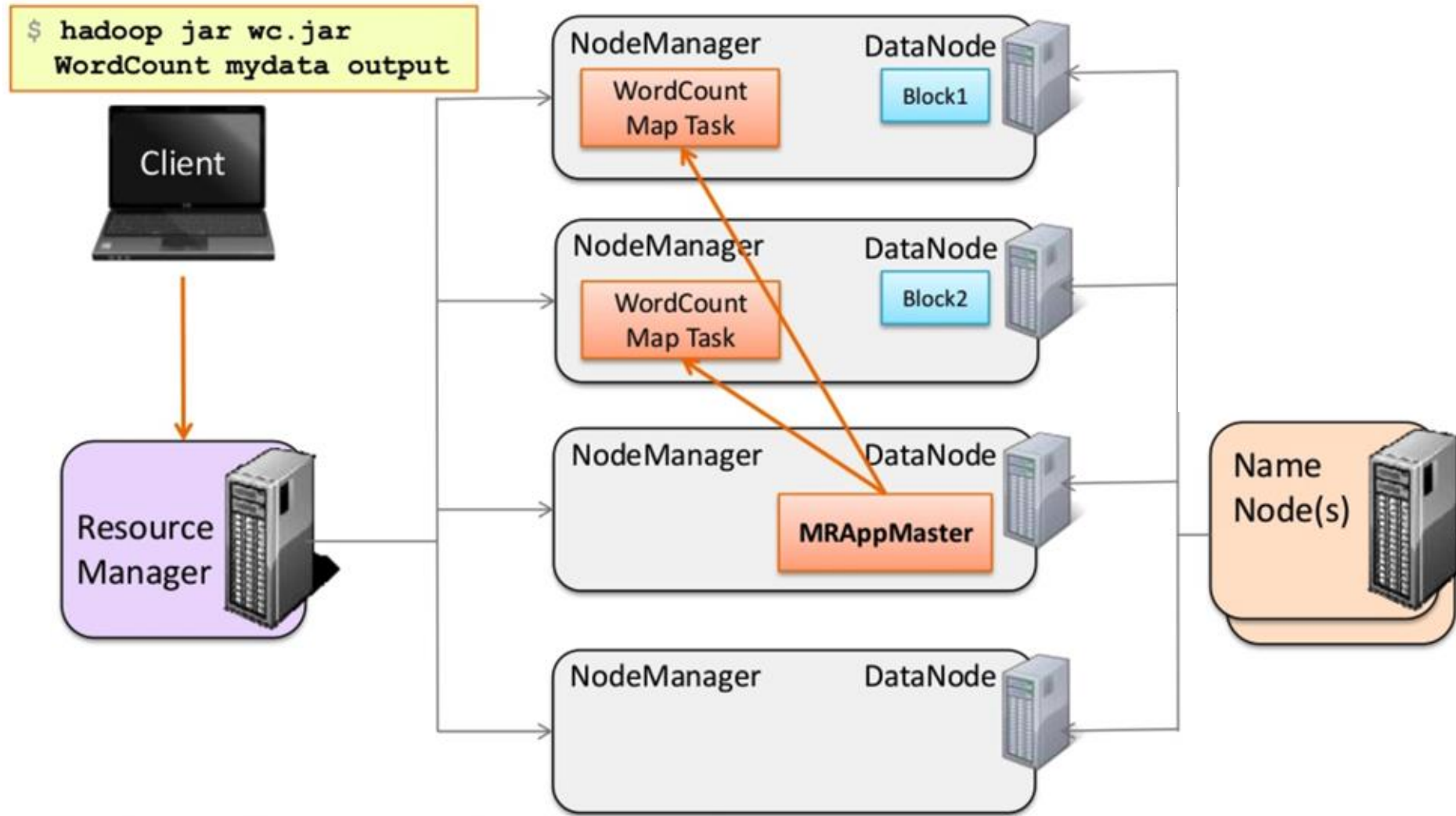


L'**ApplicationMaster** usa i metadati di HDFS (come la posizione dei blocchi) per determinare dove risiedono i dati.

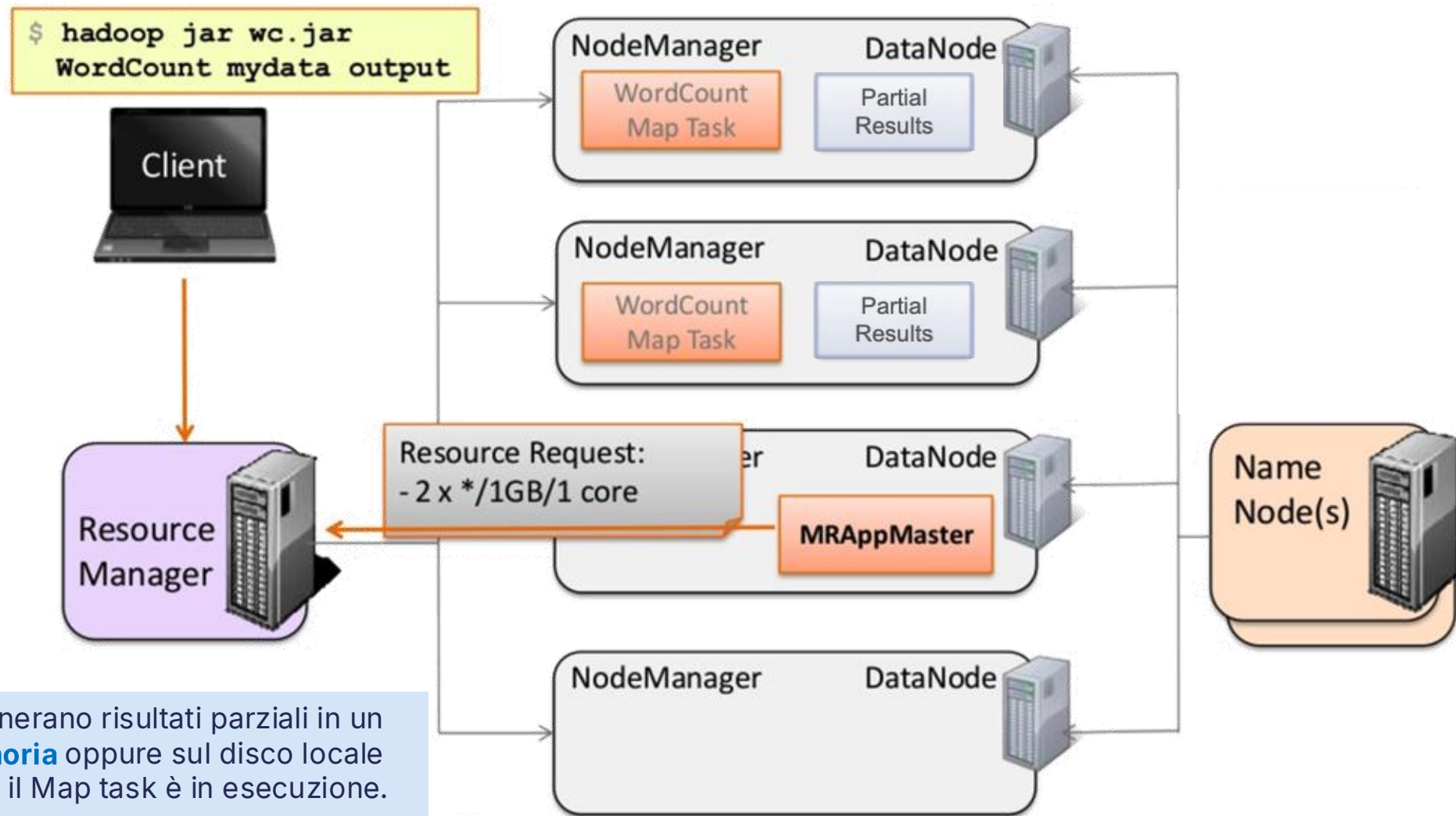
Data locality in YARN+HDFS



Data locality in YARN+HDFS

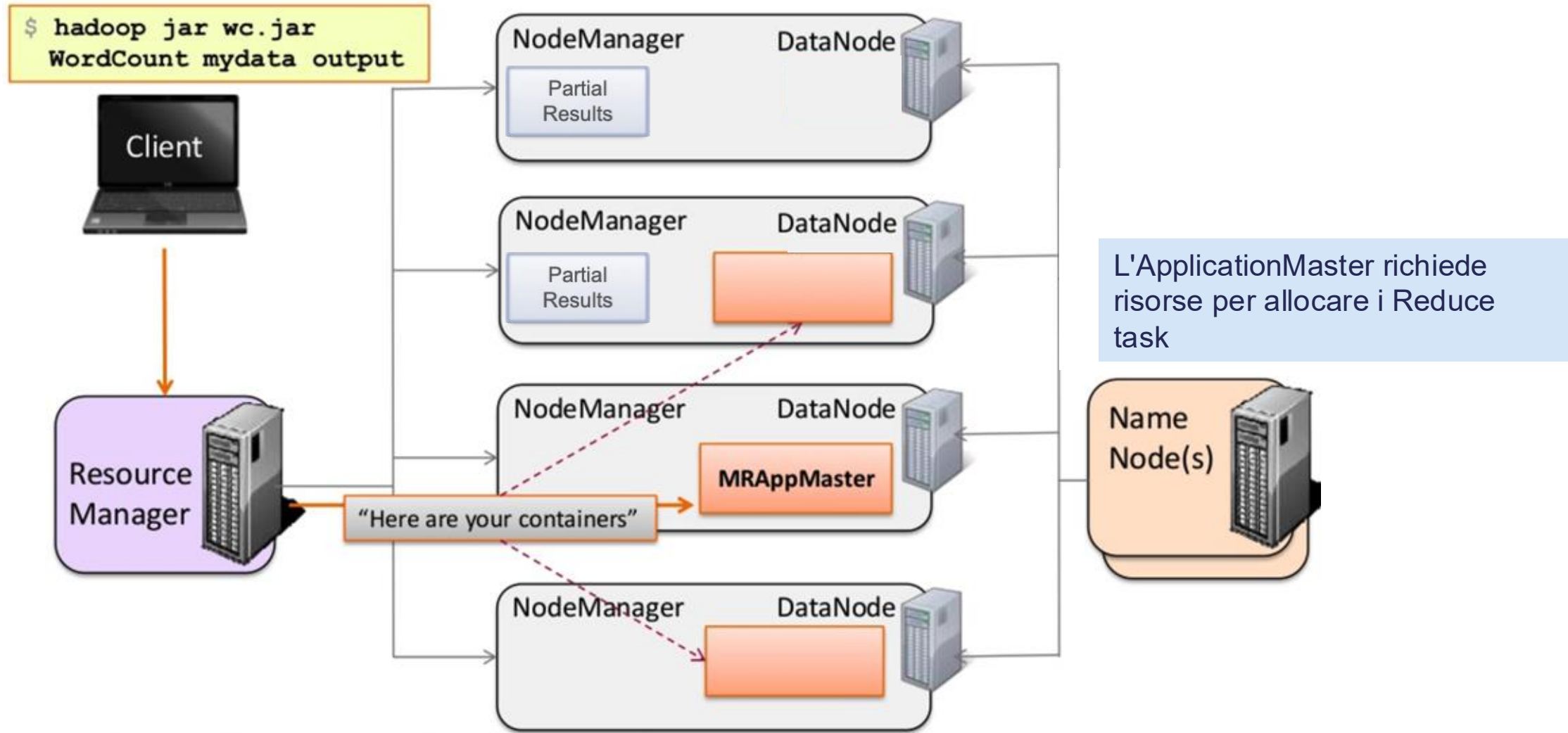


Data locality in YARN+HDFS

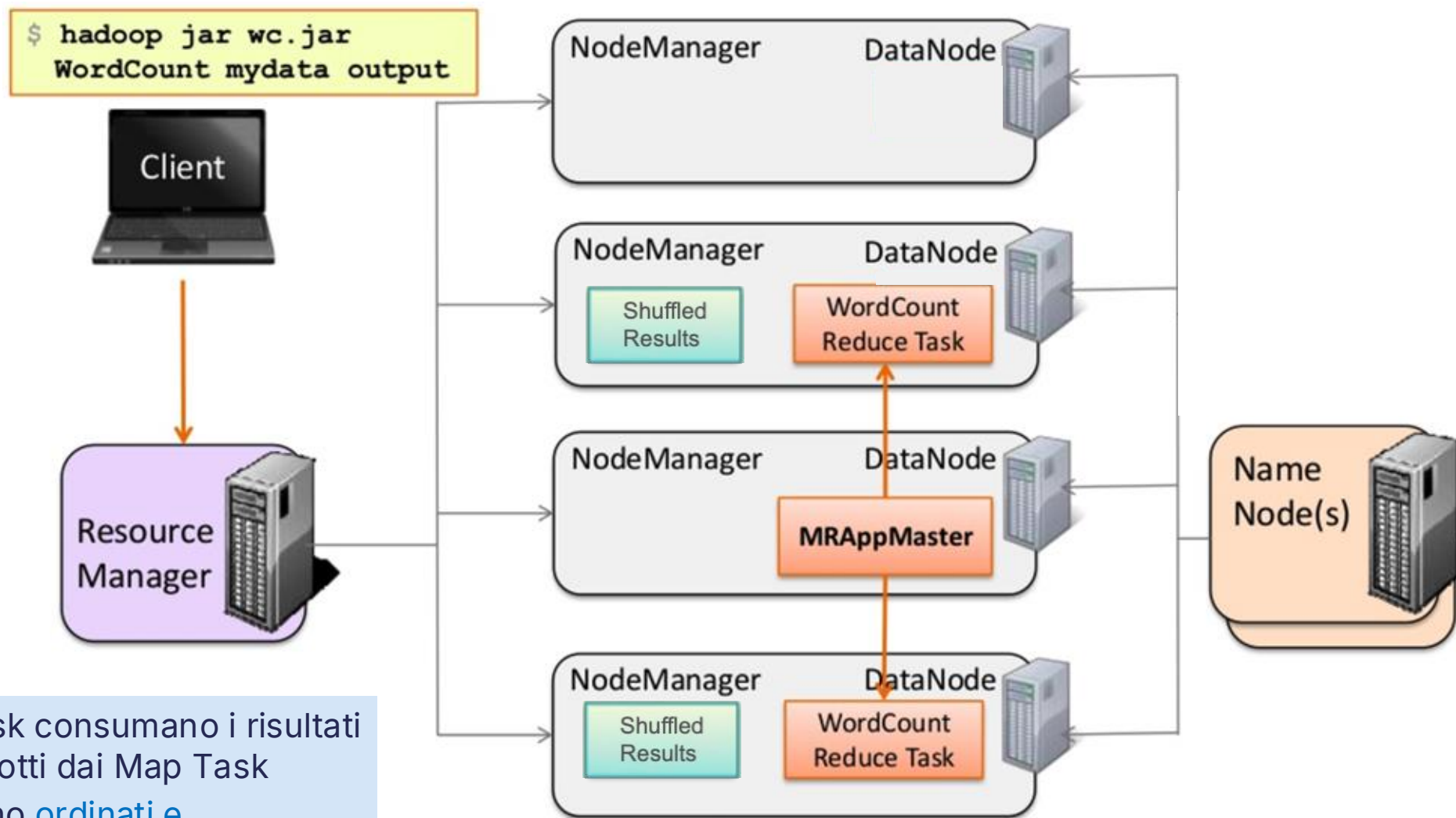


I Map Task generano risultati parziali in un **buffer in memoria** oppure sul disco locale del nodo dove il Map task è in esecuzione. I dati non vengono scritti su HDFS perché sono transitori e non necessitano di replicazione.

Data locality in YARN+HDFS

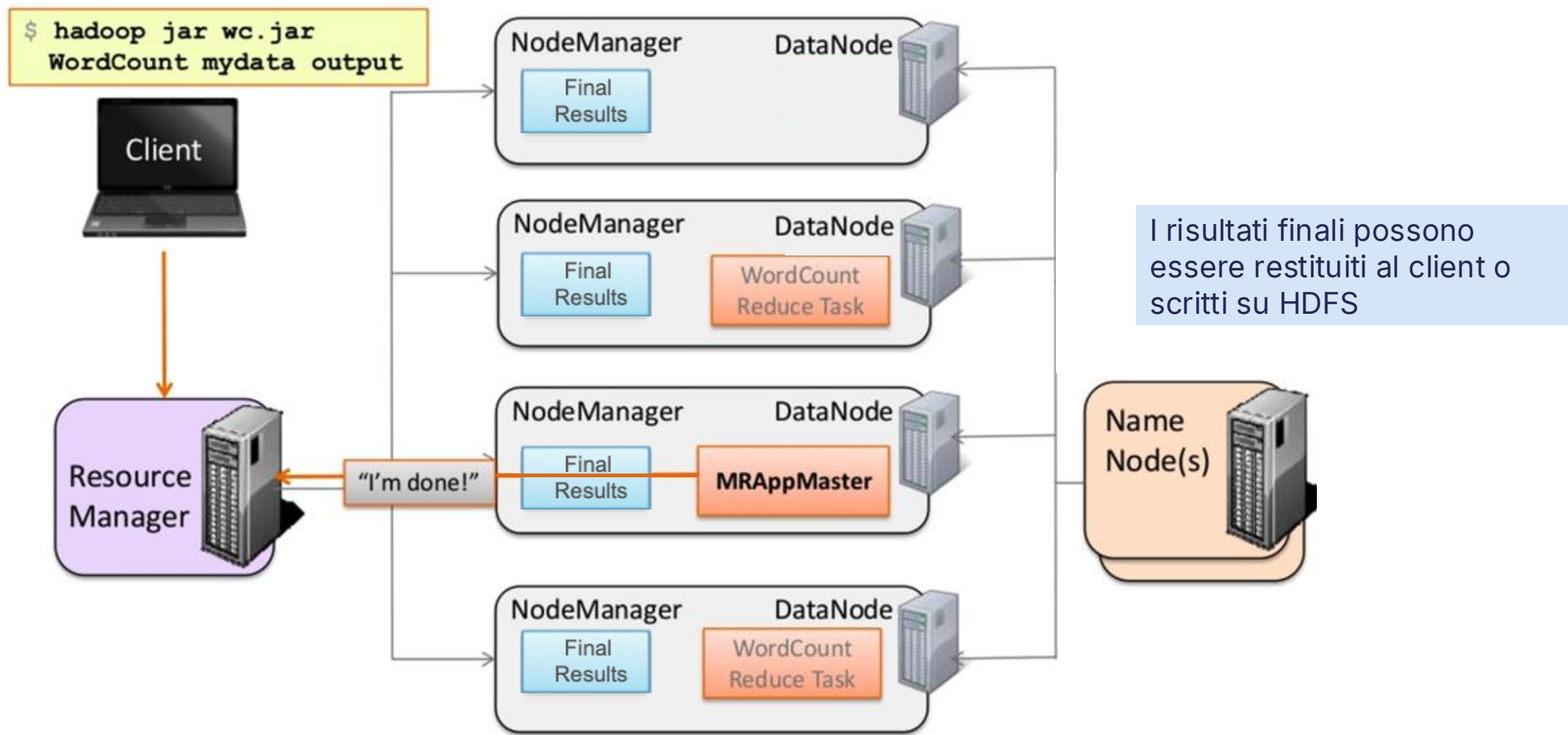


Data locality in YARN+HDFS



I Reduce Task consumano i risultati parziali prodotti dai Map Task
I dati vengono **ordinati e raggruppati per chiave** prima di essere passati alla funzione reduce

Data locality in YARN+HDFS



YARN e le forme della trasparenza

Trasparenza	Implementata	Come in YARN
Accesso	Sì	API uniformi (ApplicationMaster, ResourceManager) per richiedere risorse: il client sottomette job ignorando dove gireranno i container.
Posizione	Sì	Il ResourceManager assegna i container ai NodeManager; l'applicazione interagisce con risorse logiche, non con host fisici.
Rilocazione	No	Un container in esecuzione non viene spostato: in caso di fallimento del nodo viene riavviato altrove (preemption, non migrazione live).
Migrazione	No	YARN non sposta task attivi tra nodi; il riavvio comporta perdita dello stato locale del container.
Replica	Parziale	Repliche solo dei componenti di sistema (HA del ResourceManager con standby); i container applicativi non sono replicati.
Concorrenza	Sì	Scheduler (Capacity/Fair) gestisce richieste concorrenti di più applicazioni e utenti, isolando risorse via queue e cgroups.
Guasti	Sì	NodeManager heartbeat + retry automatico dei container falliti; ApplicationMaster recovery; HA del ResourceManager.